

# Databases 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 7 – Relational algebra and query optimization

1

1

## Motivation

The Relational model is **precise, implementable**.

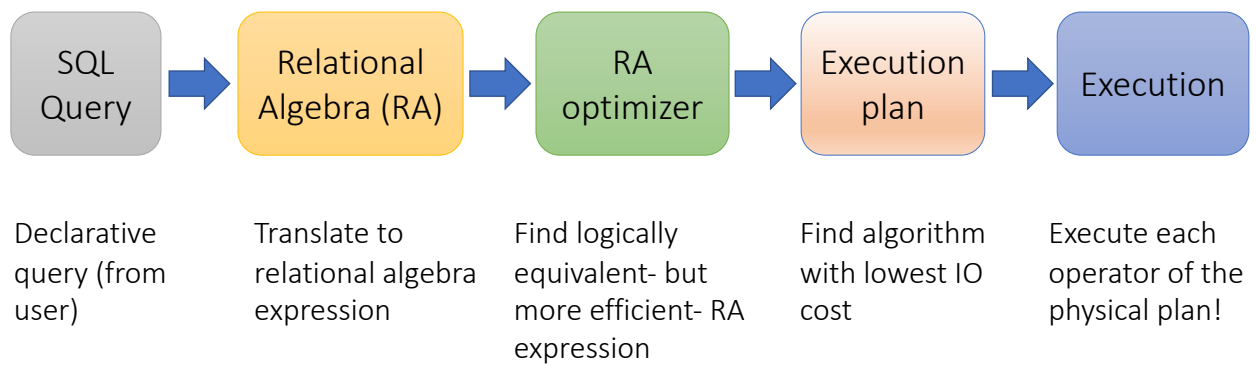
SQL is a ***declarative*** language.

Database maps internally Relational model and SQL queries into ***procedures (algorithm)***.

2

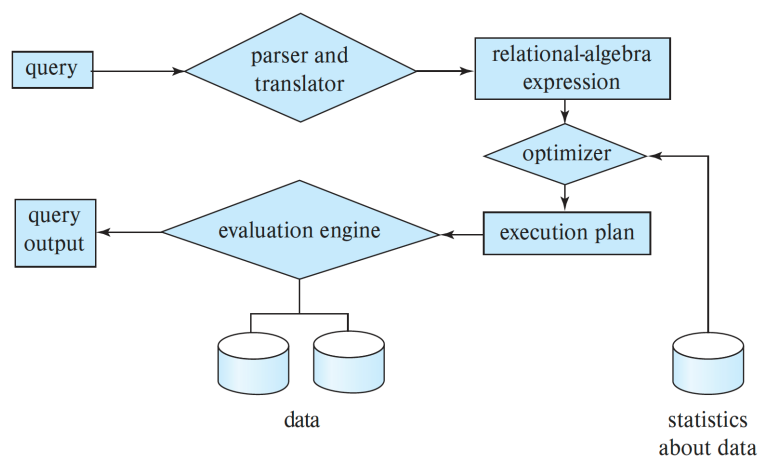
## RDBMS query processing architecture

How does a SQL engine work ?



3

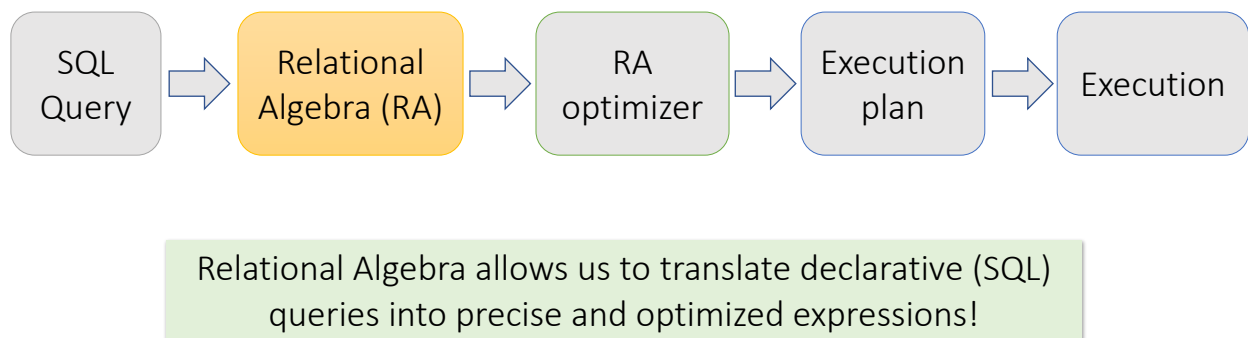
## RDBMS query processing architecture



source: Silberschatz A., Korth, HF., Sudarshan, S. (2020) Database System Concepts, 7th ed., McGraw Hill.

4

## RDBMS query processing architecture



5

## Relational algebra

The relational algebra consists of a set of operations that take one or two relations as input and produce a new relation as their result.

- **Unary operations** (operate on one relations): select, project, rename.
- **Binary operations** (operate on pairs of relations): union, Cartesian product, join, set difference.
- Formally a relation is a set of tuples, relations **cannot contain duplicate records**.
- In practice, tables in DBMSs are permitted to contain duplicates unless a specific constraint prohibits it.
- In discussing the formal relational algebra, we require that duplicates be eliminated, as is required by the mathematical definition of a set.

6

## The Selection Operation ( $\sigma$ )

- The selection operation selects tuples that satisfy a given predicate (condition).
- Notation:  $\sigma_c(R)$
- The condition  $c$  can be  $=, <, \leq, >, \geq, <>$
- Logical operators: AND, OR, NOT

Examples:

- $\sigma_{\text{price} > 4}(\text{Products})$

```
SELECT *
FROM Products
WHERE price > 4;
```

- $\sigma_{\text{maker} = \text{'ACME'} \text{ AND category} = \text{'food'}}(\text{Products})$

```
SELECT *
FROM Products
WHERE maker = 'ACME'
AND category = 'food';
```

7

## The Projection Operation ( $\Pi$ )

- The projection operation is a unary operation that returns its argument relation, with certain attributes left out.
- A relation is a set thus duplicate rows are eliminated.
- Notation:  $\Pi_{A_1, \dots, A_n}(R)$

Example:

- $\Pi_{\text{name, maker}}(\text{Product})$

- Input schema: `Products(name, maker, category, price)`

- Output schema: `Answer(name, maker)`

```
SELECT DISTINCT name, maker
FROM Products;
```

8

## Composition of Relational Operations

- Relational algebra operations can be composed together into a relational algebra expression.

`Products(name, maker, category, price)`

$\Pi_{name,price}(\sigma_{price>4}(Products))$



$\sigma_{price>4}(\Pi_{name,price}(Products))$

`SELECT DISTINCT name, price  
FROM Products  
WHERE price > 4;`

Are these logically equivalent?

9

## Composition of Relational Operations

- Here, projection & selection commute:

$\Pi_{name,price}(\sigma_{price>4}(Products)) \equiv \sigma_{price>4}(\Pi_{name,price}(Products))$

- What about here?

$\Pi_{name}(\sigma_{price>4}(Products)) \not\equiv \sigma_{price>4}(\Pi_{name}(Products))$

On the right side as a result of the projection,  
we will not have the price attribute and will  
not be able to perform a selection!

10

## The Assignment Operation ( $\leftarrow$ )

- The assignment operation, denoted by  $\leftarrow$ , works like assignment in a programming language.
- $\text{ExpensiveProducts} \leftarrow \Pi_{\text{name,price}} (\sigma_{\text{price} > 4}(\text{Products}))$

`Products(name, maker, category, price)`



`ExpensiveProducts(name, price)`

- For relational algebra queries, assignment must always be made to a temporary relation variable. Assignments to permanent relations constitute a database modification.

11

## The Rename Operation ( $\rho$ )

- Unlike relations in the database, the results of relational algebra expressions do not have a name that we can use to refer to them.
- Changes the schema, not the instance
- Notation:  $\rho_X(R)$  or  $\rho_{X(A_1, \dots, A_n)}(R)$

Examples:

- $\rho_{pr}(\text{Product})$
- $\rho_{pr(a, b, c, d)}(\text{Product})$

`Product(name, maker, category, price)`

`SELECT *`  
`FROM Product AS pr;`

`SELECT name AS a, maker AS b,`  
`category AS c, price AS d`  
`FROM Product AS pr;`

12

## The Cartesian-Product Operation ( $\times$ )

- The Cartesian-product operation, denoted by a cross ( $\times$ ), allows us to combine information from any two relations.
- Notation:  $R1 \times R2$

Example:

- $Students \times Courses$

```
SELECT *
FROM Students, Courses;
```

- Rare in practice; mainly used to express joins.

13

## The Join Operation ( $\bowtie$ )

- A (theta) join that involves a predicate:
- Notation:  $R1 \bowtie_{\theta} R2 = \sigma_{\theta}(R1 \times R2)$
- Here  $\theta$  can be any condition
- $\theta$  with  $=$  is called **equi join**

Example:

- $\sigma_{Students.SID=Enrolled.SID}(Students \times Enrolled)$
- $Students \bowtie_{Students.SID=Enrolled.SID} Enrolled$

14

## Example: Converting Query to RA expression

```
SELECT DISTINCT
  s.first_name
  s.last_name
  e.grade
FROM Students s
JOIN Enrolled AS e
  ON s.SID = e.SID
WHERE e.grade > 3.5;
```



$$\Pi_{s.first\_name, s.last\_name, e.grade}(\sigma_{grade > 3.5}(\rho_s(Students) \bowtie_{s.SID=e.SID} \rho_e(Enrolled)))$$

15

## Semijoin ( $\bowtie$ ) ( $\ltimes$ )

- Notation:  $R \ltimes S = \Pi_{A_1, \dots, A_n}(R \bowtie S)$
- Where  $A_1, \dots, A_n$  are the attributes in  $R$

Example:

- $Students \ltimes_{Students.SID = Enrolled.SID} Enrolled$

```
SELECT DISTINCT
  Students.SID
  Students.first_name
  Students.last_name
FROM Students
JOIN Enrolled
  ON Students.SID = Enrolled.SID;
```

```
SELECT DISTINCT
  Students.SID
  Students.first_name
  Students.last_name
FROM Students
WHERE EXISTS(
  SELECT *
  FROM Enrolled
  WHERE Students.SID = Enrolled.SID);
```

16

## Antijoin (antisemijoin) ( $\bar{\bowtie}$ )

- Notation  $R \bar{\bowtie} S = R - R \bowtie S$

Example:

- $\text{Students} \bar{\bowtie}_{\text{Students.SID} = \text{Enrolled.SID}} \text{Enrolled}$

```
SELECT DISTINCT
  Students.SID
  Students.first_name
  Students.last_name
FROM Students
WHERE NOT EXISTS(
  SELECT *
  FROM Enrolled
  WHERE Students.SID = Enrolled.SID);
```

17

## Set operations

Union:  $R \cup S$

- Example:  $\text{Students} \cup \text{Professors}$

Difference  $R - S$

- Example:  $\text{Persons} - \text{Professors}$

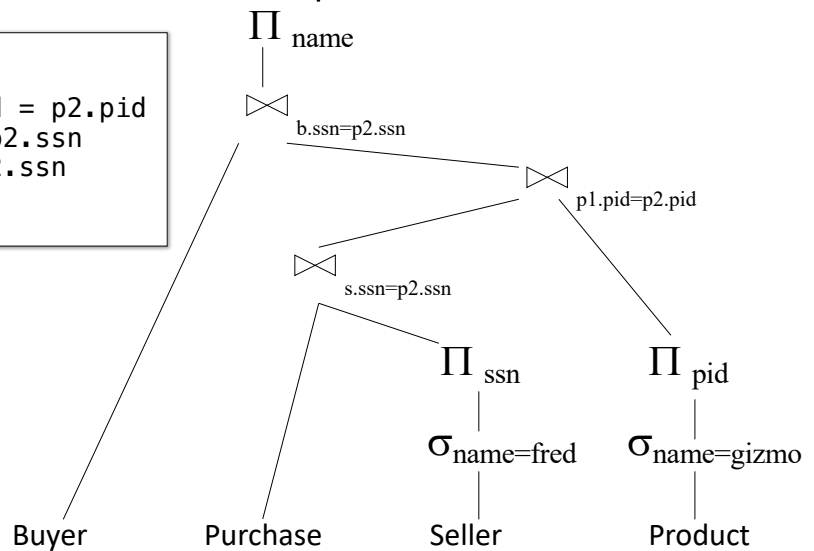
Intersection:  $R \cap S = R - (R - S)$

- It is a derived operator
- Also, can be expressed as a join!

18

## RA Expressions Can Get Complex!

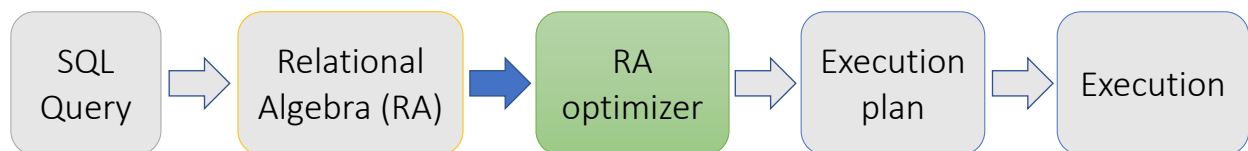
```
SELECT DISTINCT name
FROM Product p1
JOIN Purchase p2 ON p1.pid = p2.pid
JOIN Seller s ON s.ssn = p2.ssn
JOIN Buyer b ON b.ssn = p2.ssn
WHERE p2.name='fred'
      AND p1.name='gizmo';
```



19

## RDBMS query processing architecture

Logical Optimization



We'll look at how to optimize Relational Algebra expression now

20

## Measures of Query Cost

- The cost of query evaluation can be measured in terms of:
  - **Disk accesses (I/O operations)**
  - **CPU time** to execute a query
  - **Cost of communication**, for parallel and distributed database systems
- The cost model used by PostgreSQL (in 2018)
  - a CPU cost per tuple,
  - a CPU cost for processing each index entry (in addition to the I/O cost),
  - a CPU cost per operator or function (such as arithmetic operators, comparison operators, and related functions).

21

## Query optimization

- **Query optimization** is the process of selecting the most efficient query-evaluation plan.
- We do not expect users to write their queries so that they can be processed efficiently. Rather, we expect the system to construct a query-evaluation plan that minimizes the cost of query evaluation.
- Optimization occurs at the relational-algebra level, where the system attempts to find an expression that is **equivalent to the given expression**, but more efficient to execute.
- Another aspect is selecting a detailed strategy for processing the query, such as choosing the algorithm to use for executing an operation, choosing the specific indices to use.

22

## Query optimization guidelines

- Cost-based query optimizers compute the cost of each alternative and pick the least cost alternative.
- The cost of an expression is computed based on statistics.
- Alternatives are generated by enumerating equivalent expressions.
- Additional rules are used to limit number of investigated equivalent expressions.
- Exemplary basic rules:
  - Perform selection operations as early as possible
  - Perform projections early
  - Avoid Cartesian products
  - Joins can be re-ordered

23

## Relational algebra equivalence rules

1.  $\sigma_A(R) \equiv \sigma_A \sigma_A(R)$
2.  $\sigma_A \sigma_B(R) \equiv \sigma_B \sigma_A(R)$
3.  $\sigma_{A \wedge B}(R) \equiv \sigma_A(\sigma_B(R)) = \sigma_B(\sigma_A(R))$
4.  $\sigma_{A \vee B}(R) \equiv \sigma_A(R) \cup \sigma_B(R)$
5.  $\sigma_A(R \times P) \equiv \sigma_{B \wedge C \wedge D}(R \times P) = \sigma_D(\sigma_B(R) \times \sigma_C(P))$
6.  $\sigma_A(R \cup P) \equiv \sigma_A(R) \cup \sigma_A(P)$
7.  $\sigma_A(R \cap P) \equiv \sigma_A(R) \cap \sigma_A(P) = \sigma_A(R) \cap P = R \cap \sigma_A(P)$
8.  $\sigma_A(R \setminus P) \equiv \sigma_A(R) \setminus \sigma_A(P) = \sigma_A(R) \setminus P$

24

## Relational algebra equivalence rules

9.  $\Pi_{a_1, \dots, a_n}(\sigma_A(R)) \equiv \sigma_A(\Pi_{a_1, \dots, a_n}(R))$
10.  $\Pi_{a_1, \dots, a_n}(\Pi_{b_1, \dots, b_n}(R)) \equiv \Pi_{a_1, \dots, a_n}(R) \iff \{a_1, \dots, a_n\} \subseteq \{b_1, \dots, b_n\}$
11.  $\Pi_{a_1, \dots, a_n}(R \cup P) \equiv \Pi_{a_1, \dots, a_n}(R) \cup \Pi_{a_1, \dots, a_n}(P)$
12.  $(R_1 \bowtie_{\theta_1} R_2) \bowtie_{\theta_2 \wedge \theta_3} R_3 \equiv R_1 \bowtie_{\theta_1 \wedge \theta_3} (R_2 \bowtie_{\theta_2} R_3)$
13.  $\sigma_{\theta_1}(R_1 \bowtie_{\theta} R_2) \equiv (\sigma_{\theta_1}(R_1)) \bowtie_{\theta} R_2$
14.  $\sigma_{\theta_1 \wedge \theta_2}(R_1 \bowtie_{\theta} R_2) \equiv (\sigma_{\theta_1}(R_1)) \bowtie_{\theta} (\sigma_{\theta_2}(R_2))$
15.  $\Pi_{A_1 \cup A_2}(R_1 \bowtie_{\theta} R_2) \equiv (\Pi_{A_1}(R_1)) \bowtie_{\theta} (\Pi_{A_2}(R_2))$
16. and many more...

25

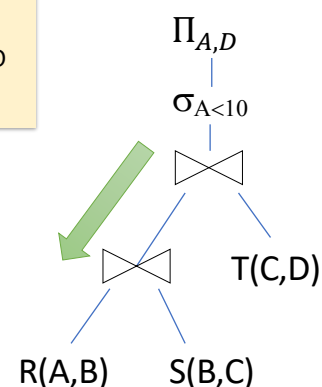
## Relational algebra optimizer example

$R(A,B)$   $S(B,C)$   $T(C,D)$

```
SELECT R.A, T.D
FROM R
JOIN S ON R.B = S.B
JOIN T ON S.C = T.C
WHERE R.A < 10;
```

Push down  
selection on A so  
it occurs earlier

$\Pi_{A,D}(\sigma_{A < 10}(T \bowtie (R \bowtie S)))$



Rule #13:  $\sigma_{\theta_1}(R_1 \bowtie_{\theta} R_2) \equiv (\sigma_{\theta_1}(R_1)) \bowtie_{\theta} R_2$

26

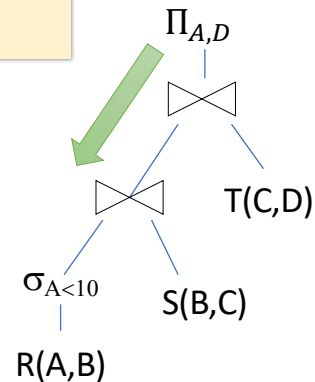
## Relational algebra optimizer example

$R(A,B)$   $S(B,C)$   $T(C,D)$

```
SELECT R.A, T.D
FROM R
JOIN S ON R.B = S.B
JOIN T ON S.C = T.C
WHERE R.A < 10;
```

Push down  
projection so it  
occurs earlier

$\Pi_{A,D}(T \bowtie (\sigma_{A < 10}(R) \bowtie S))$



Rule #15:  $\Pi_{A_1 \cup A_2}(R_1 \bowtie_{\theta} R_2) \equiv (\Pi_{A_1}(R_1)) \bowtie_{\theta} (\Pi_{A_2}(R_2))$

27

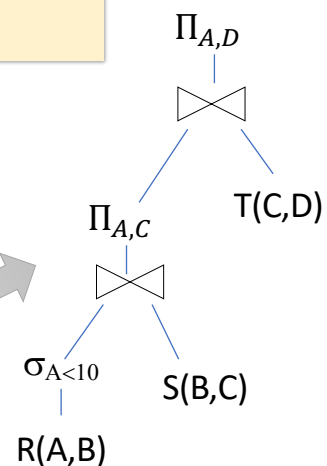
## Relational algebra optimizer example

$R(A,B)$   $S(B,C)$   $T(C,D)$

```
SELECT R.A, T.D
FROM R
JOIN S ON R.B = S.B
JOIN T ON S.C = T.C
WHERE R.A < 10;
```

We eliminate B  
earlier!

$\Pi_{A,D}(T \bowtie \Pi_{A,C}(\sigma_{A < 10}(R) \bowtie S))$



28

## Query Transformations

### OR expansion

```
SELECT first_name, last_name
FROM Students
WHERE first_name = 'Mia' OR last_name = 'Vega';
```



```
SELECT first_name, last_name
FROM Students
WHERE first_name = 'Mia'
UNION ALL
SELECT first_name, last_name
FROM Students
WHERE last_name = 'Vega';
```

29

## Query Transformations

### Subquery unnesting and view merging

```
2 SELECT e.CID, e.grade
   FROM Enrolled e
  WHERE CID IN (
1  SELECT CID
   FROM Courses
  WHERE ECTS >= 5);
```

=

```
SELECT e.CID, e.grade
FROM Enrolled e
RIGHT JOIN Courses c ON c.CID = e.CID
WHERE c.ECTS >= 5;
```

30

## Statistics – catalog Information

The database-system catalog stores the following statistical information about database relations:

- The number of tuples in the relation.
- The number of blocks containing tuples of relation.
- The size of a tuple of relation in bytes.
- The number of tuples of relation that fit into one block.
- The number of distinct values that appear in the relation for attribute A.
- Statistics about the **heights** of B<sup>+</sup> tree indices and **number of leaf** pages in the indices.

31

## Statistics – updating

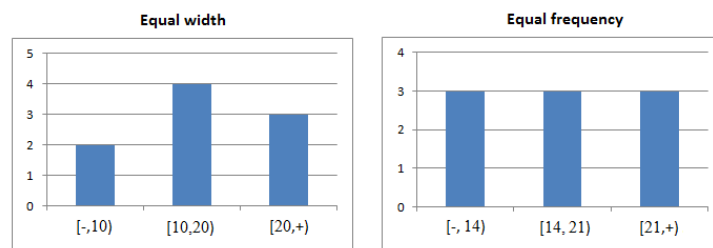
- Accurate statistics requires an update every time a relation is modified. This update incurs a substantial amount of overhead.
- Most systems do not update the statistics on every modification. Instead, they update the statistics during periods of light system load. As a result, the statistics used for choosing a query-processing strategy may not be completely accurate.
- Many database systems (IBM DB2, Oracle, SQL Server) update statistics automatically at certain points of time.
- For example, the system can keep approximate track of how many tuples there are in a relation and recompute statistics if this number changes significantly.
- Another approach is to compare estimated cardinalities of a relation scan with actual cardinalities when a query is executed, and if they differ significantly, initiate an update of statistics for that relation.

32

## Statistics – histograms

Most databases store the distribution of values for each attribute as a histogram.

- **An equi-width histogram** since it divides the range of values into equal-sized ranges.
- **An equi-depth histogram** adjusts the boundaries of the ranges such that each range has the same number of values.



33

## Manual statistics update

- Keep table statistics up to date, so the optimizer has the information needed to construct an efficient execution plan.

```
ANALYZE TABLE table
  UPDATE HISTOGRAM ON attribute1, attribute2
  WITH 10 BUCKETS;
```

34

## Oracle – Expression Statistics Store (ESS)

The database uses the ESS to determine whether an expression is “hot” (frequently accessed), thus a candidate for an In-Memory expression. During a hard parse of a query, the ESS looks for active expressions in the SELECT list, WHERE clause, GROUP BY clause, and so on.

For each segment, the ESS maintains expression statistics such as the following:

- Frequency of execution
- Cost of evaluation
- Timestamp evaluation

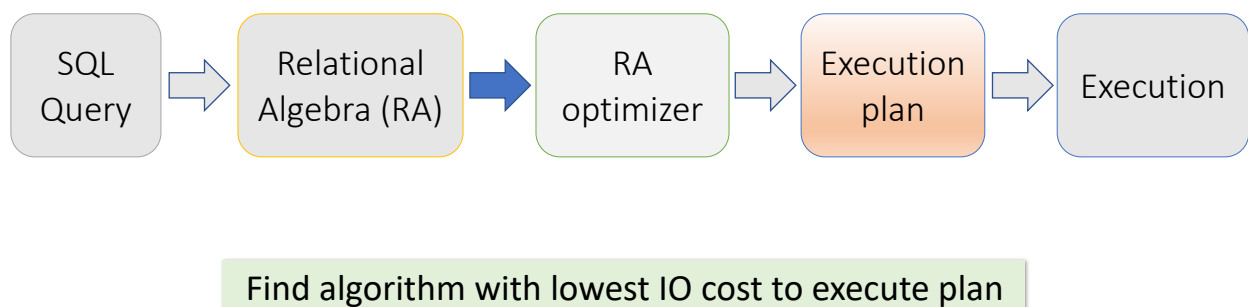
The optimizer assigns each expression a weighted score based on cost and the number of times it was evaluated. The values are approximate rather than exact. More active expressions have higher scores. The ESS maintains an internal list of the most frequently accessed expressions.

The ESS resides in the memory and also persists on disk. The database saves the statistics to disk every 15 minutes or immediately.

Source: Oracle, SQL Tuning Guide, 2022

35

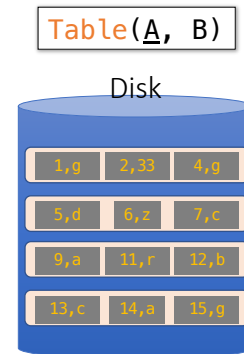
## RDBMS query processing architecture



36

## Selection operation

- In a **linear search**, the system scans each file block and tests all records to see whether they satisfy the selection condition.
- For an **equality comparison** on a **key attribute** with a **clustering index**, we can use the index to retrieve a single record that satisfies the corresponding equality condition.
- We can retrieve multiple records by using a clustering index when the selection condition specifies an equality comparison on a non-key attribute, A.



37

## Selection operation

### Algorithm

Linear Search

```
SELECT * FROM Table WHERE B >= 'e';
```

Linear Search, Equality on Key

```
SELECT * FROM Table WHERE A = 9;
```

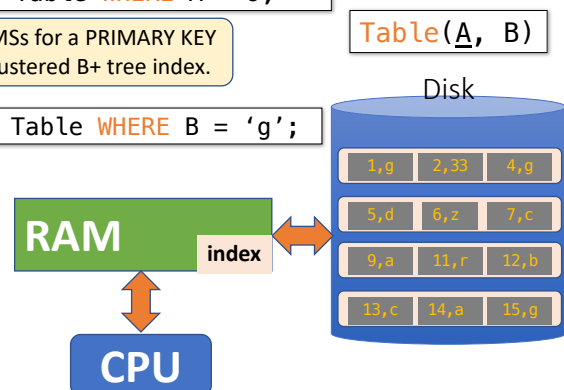
Clustering B<sup>+</sup> tree Index, Equality on Key

Many DBMSs for a PRIMARY KEY creates clustered B+ tree index.

Clustering B<sup>+</sup> tree Index, Equality on Non-keySecondary B<sup>+</sup> tree Index, Equality on KeySecondary B<sup>+</sup> tree Index, Equality on Non-key

```
SELECT * FROM Table WHERE B = 'g';
```

```
CREATE INDEX idxTableB ON Table (B);
```



38

## Selection operation

| Algorithm                                                 | Cost                                |
|-----------------------------------------------------------|-------------------------------------|
| Linear Search                                             | $t_s + b_r * t_T$                   |
| Linear Search, Equality on Key                            | on average $t_s + (1/2b_r) * t_T$   |
| Clustering B <sup>+</sup> tree Index, Equality on Key     | $(h_i + 1) * (t_s + t_T)$           |
| Clustering B <sup>+</sup> tree Index, Equality on Non-key | $h_i * (t_s + t_T) + t_s + b * t_T$ |
| Secondary B <sup>+</sup> tree Index, Equality on Key      | $(h_i + 1) * (t_s + t_T)$           |
| Secondary B <sup>+</sup> tree Index, Equality on Non-key  | $(h_i + n) * (t_s + t_T)$           |

$t_s$  – average seek time (disk seek time plus rotational latency)

$t_T$  – average transfer time

$b_r$  – number of blocks (all blocks)

$h_i$  – B<sup>+</sup> tree height

$n$  – number of records to fetch

$b$  – the number of blocks containing records with the specified search key

$n$  can be high

39

## Sorting

- SQL queries can specify that the output be sorted.
- Several of the relational operations, such as joins, can be implemented efficiently if the input relations are first sorted.
- We can sort a relation by building an index on the sort key and then using that index to read the relation in sorted order, but it leads to a disk access (disk seek plus block transfer) for each record, which can be very expensive, since the number of records can be much larger than the number of blocks. For this reason, it may be desirable to order the records physically.

40

## Sorting

- Data requested from database in sorted order is **extremely common**.
- Sorting is useful for eliminating *duplicate copies*.
  - Identical tuples will appear adjacent to each other as a result of sorting, and all but one copy can be removed.
- Sorting is useful for *aggregations*.
  - Identical tuples will appear adjacent to each other as a result of sorting, we gather them into groups and apply the aggregation operations on each group to get the result.
- Sorting is useful for *set operations*: *union, intersection, set-difference*.
  - First sorting both relations, and then scanning once through each of the sorted relations to produce the result.

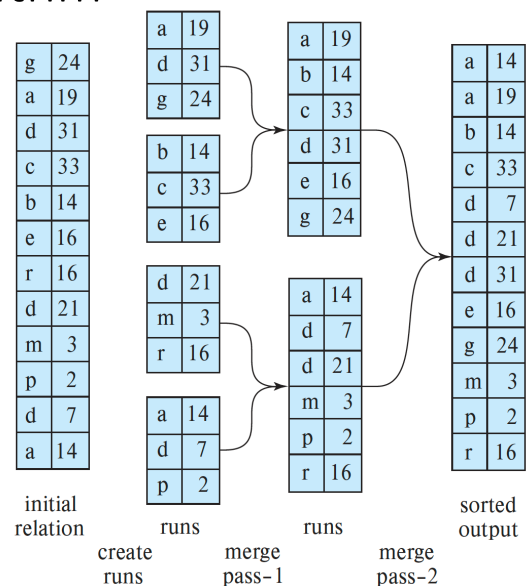
41

## External Sort-Merge Algorithm

Sorting of relations that do not fit in memory is called external sorting.

Sorting is executed in two steps:

1. Divide the unsorted list into n sublists and sort in memory.
2. Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining.



source: Silberschatz A., Korth, HF., Sudarshan, S. (2020)  
Database System Concepts, 7th ed., McGraw Hill.

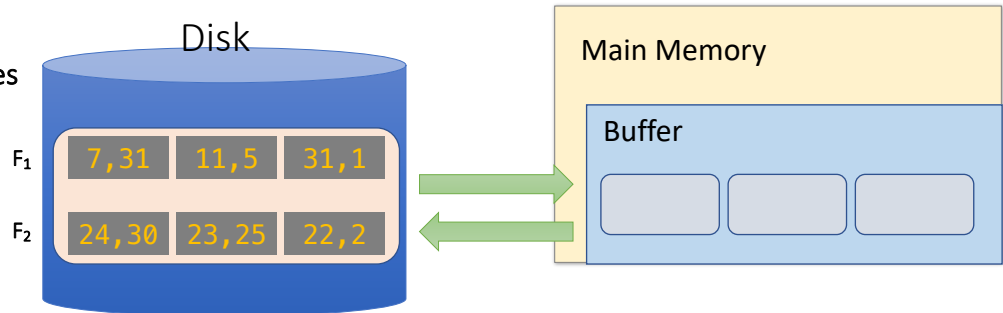
42

## External Sort-Merge Algorithm

Example:

- 3 Buffer pages
- 6-page file

Orange file  
= unsorted



1. Split into chunks small enough to **sort in memory**

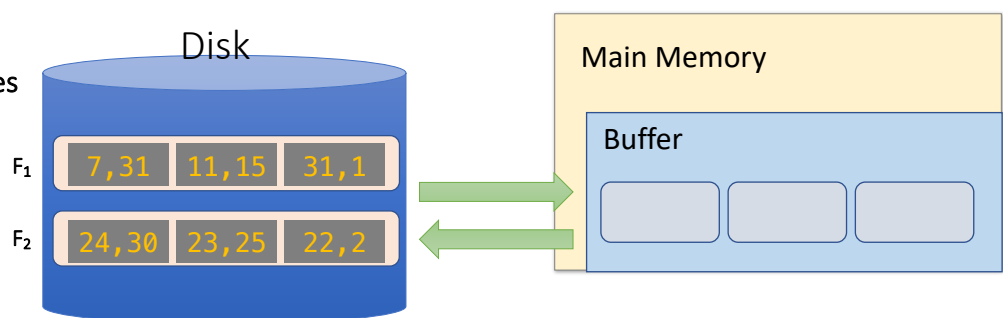
43

## External Sort-Merge Algorithm

Example:

- 3 Buffer pages
- 6-page file

Orange file  
= unsorted



1. Split into chunks small enough to **sort in memory**

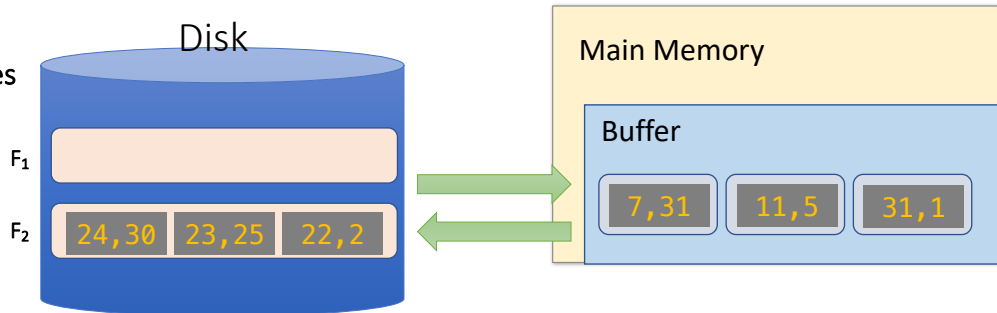
44

## External Sort-Merge Algorithm

Example:

- 3 Buffer pages
- 6-page file

Orange file  
= unsorted



1. Split into chunks small enough to **sort in memory**

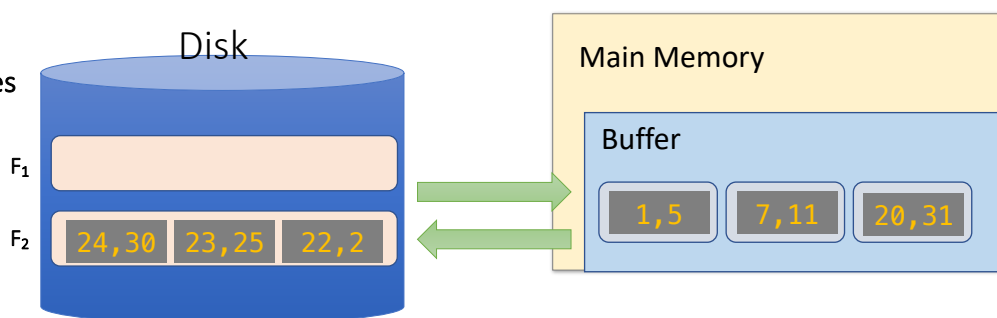
45

## External Sort-Merge Algorithm

Example:

- 3 Buffer pages
- 6-page file

Orange file  
= unsorted



1. Split into chunks small enough to **sort in memory**

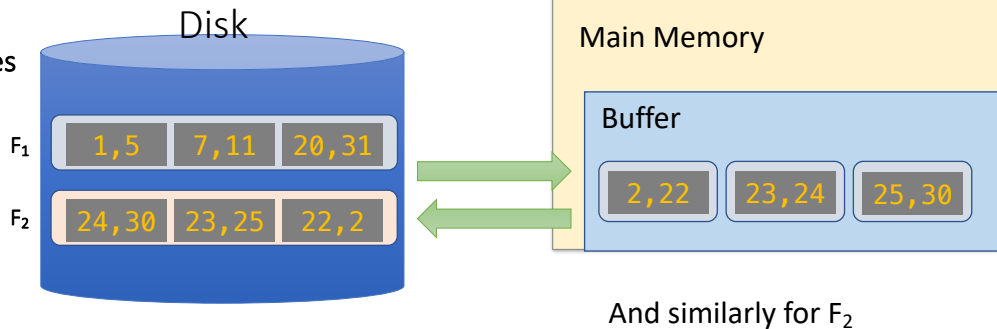
46

## External Sort-Merge Algorithm

Example:

- 3 Buffer pages
- 6-page file

Each sorted file is called a *run*



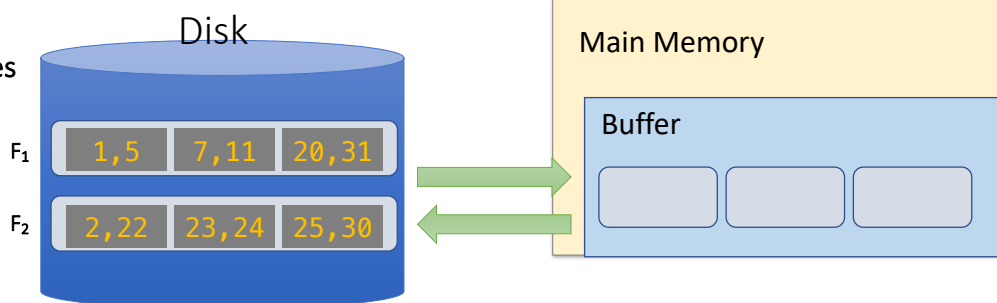
1. Split into chunks small enough to **sort in memory**

47

## External Sort-Merge Algorithm

Example:

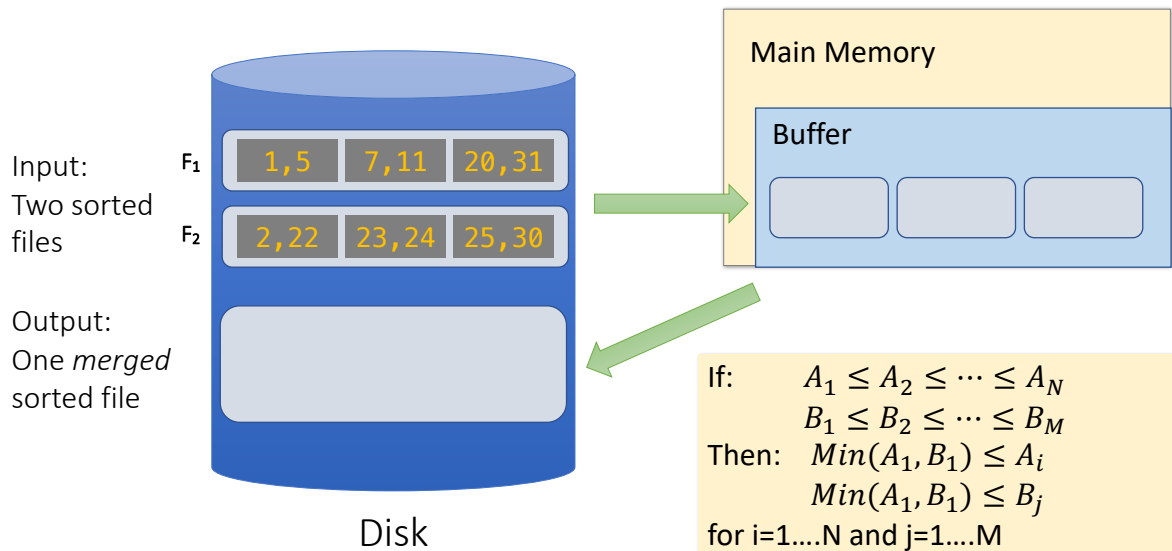
- 3 Buffer pages
- 6-page file



2. Now just run the **external merge**

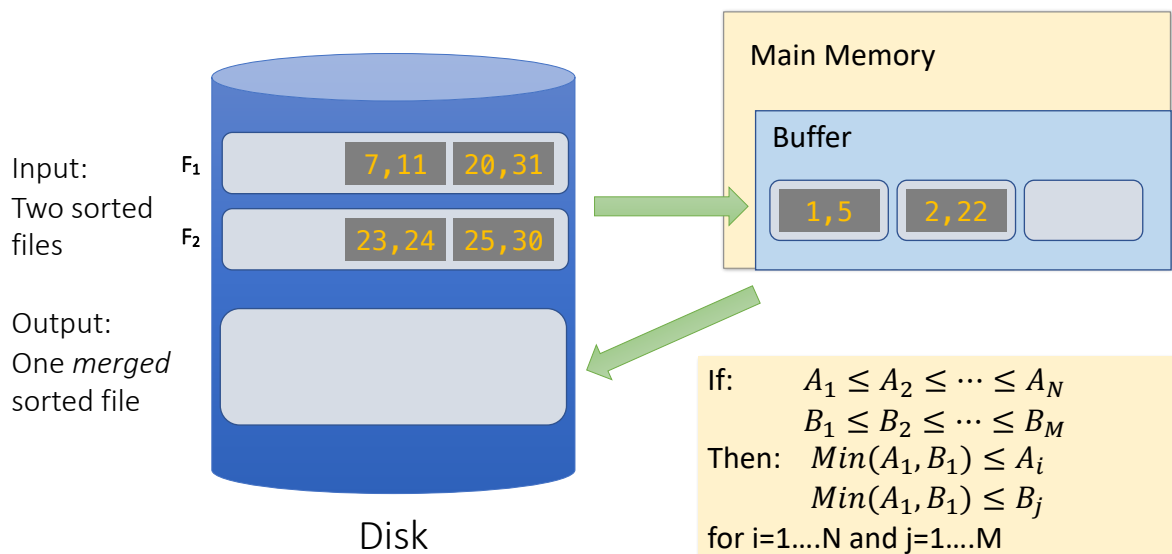
48

## External Sort-Merge Algorithm



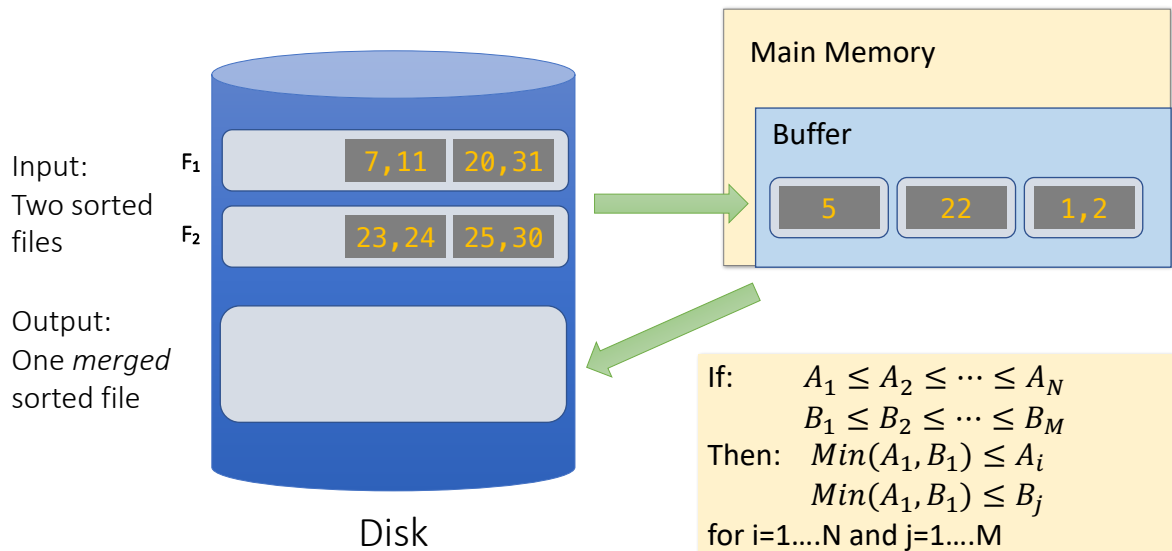
49

## External Sort-Merge Algorithm



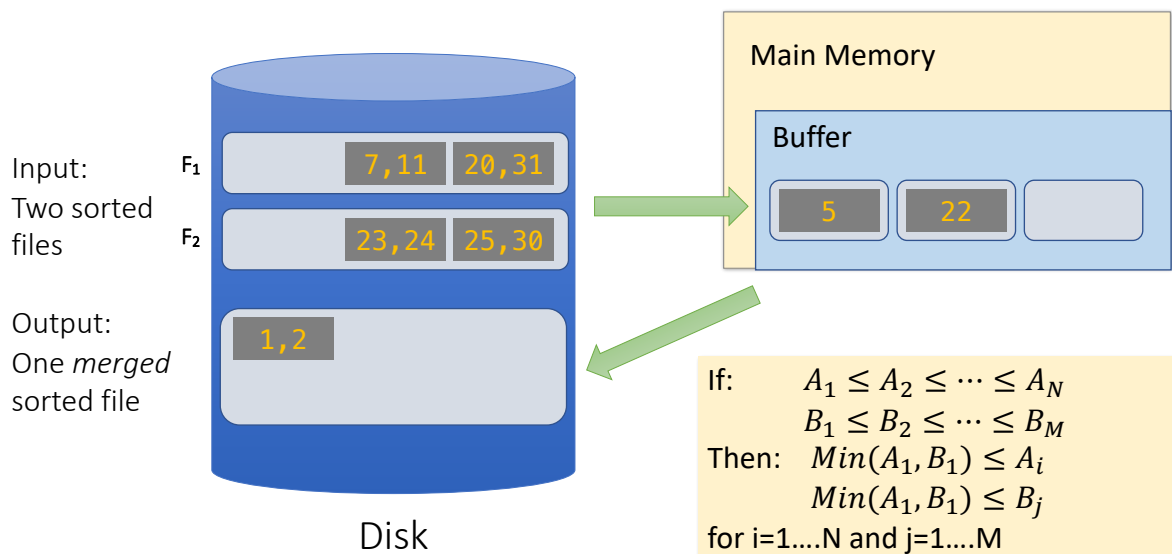
50

## External Sort-Merge Algorithm



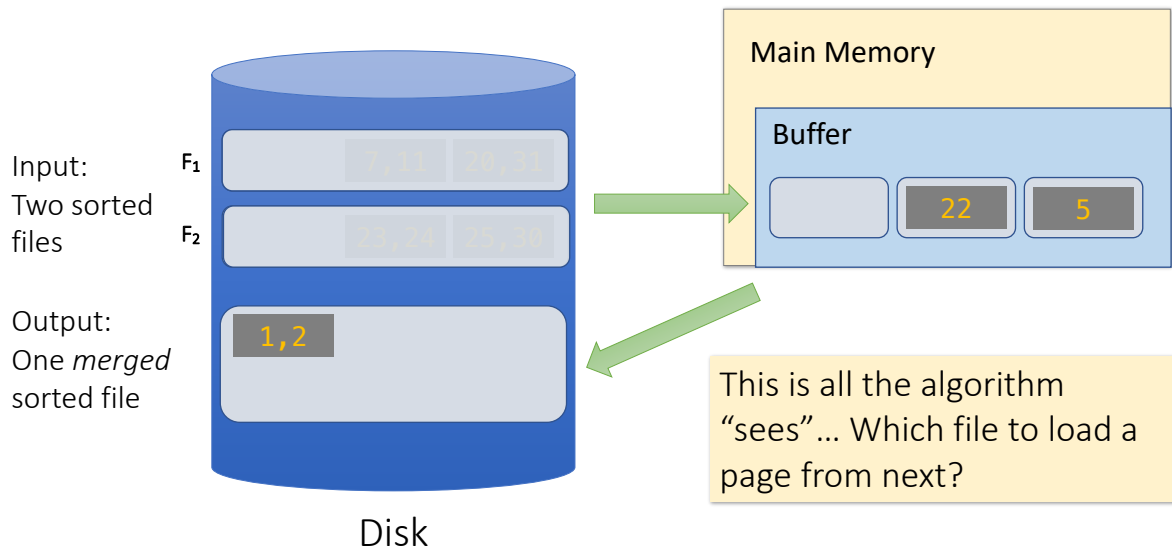
51

## External Sort-Merge Algorithm



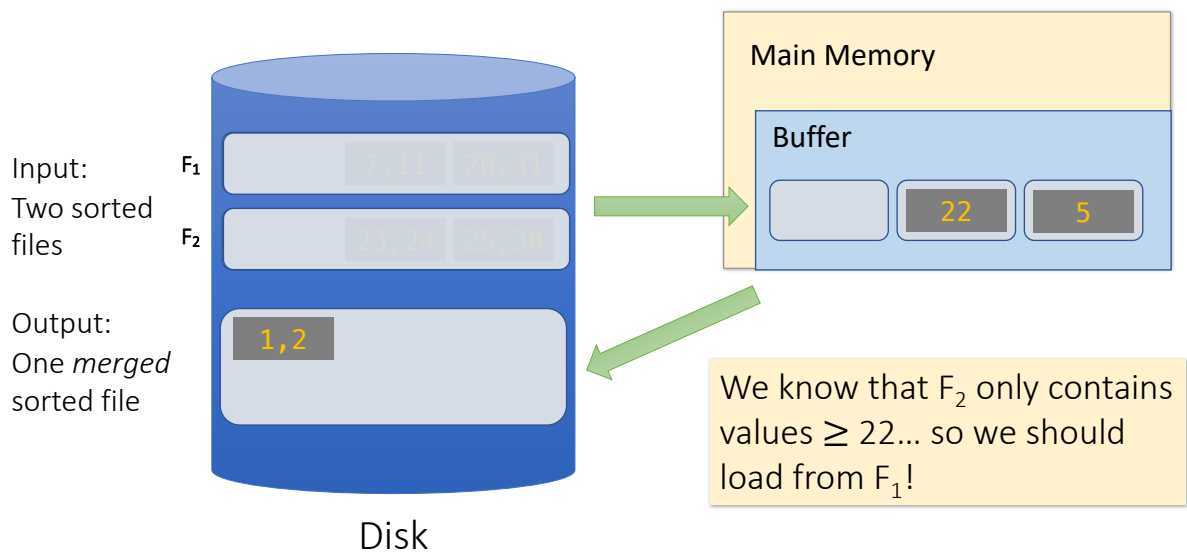
52

## External Sort-Merge Algorithm



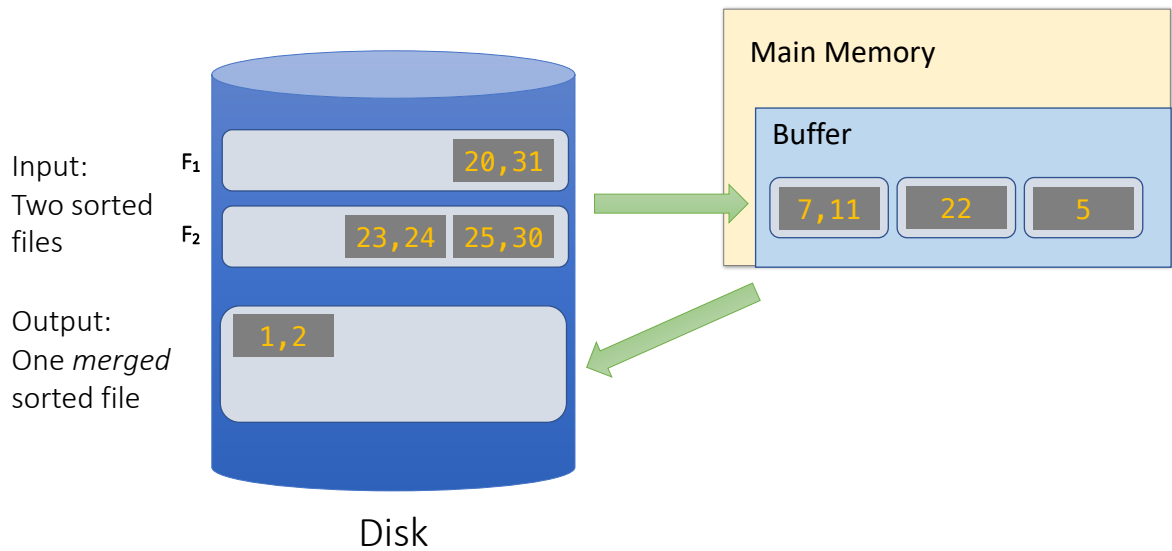
53

## External Sort-Merge Algorithm



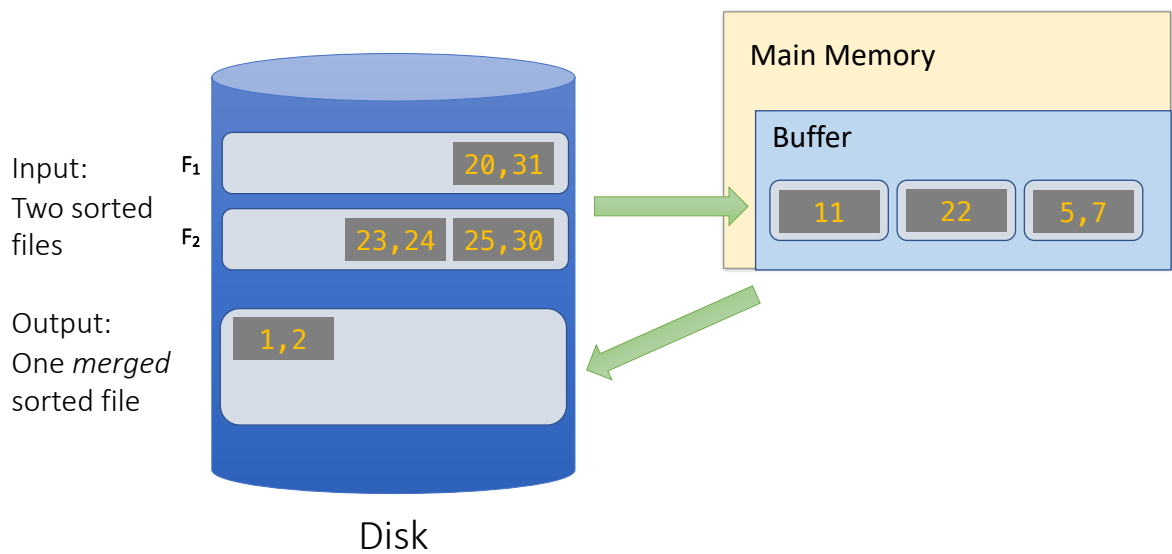
54

## External Sort-Merge Algorithm



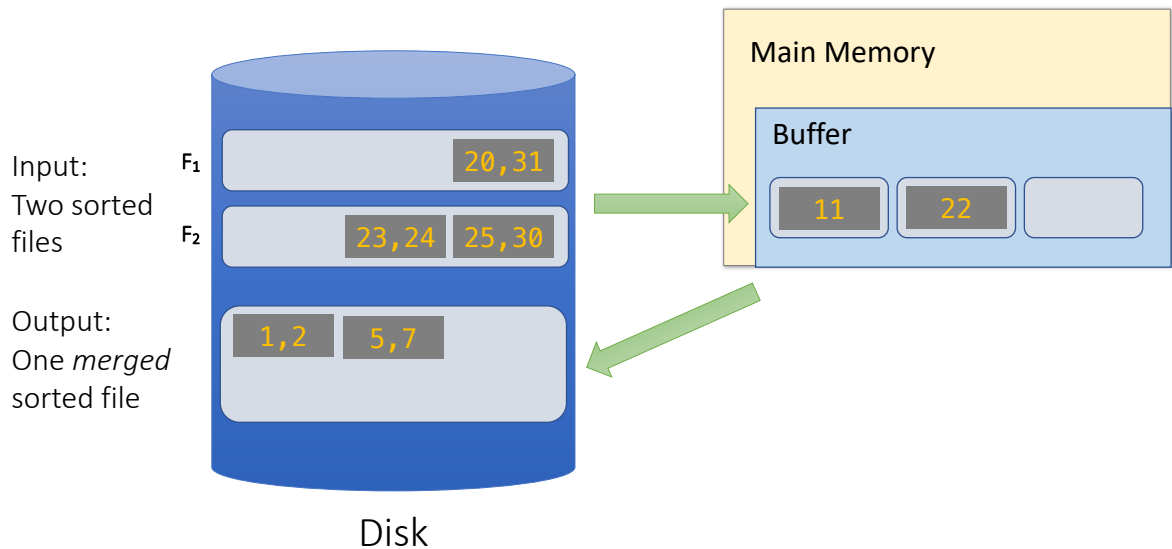
55

## External Sort-Merge Algorithm



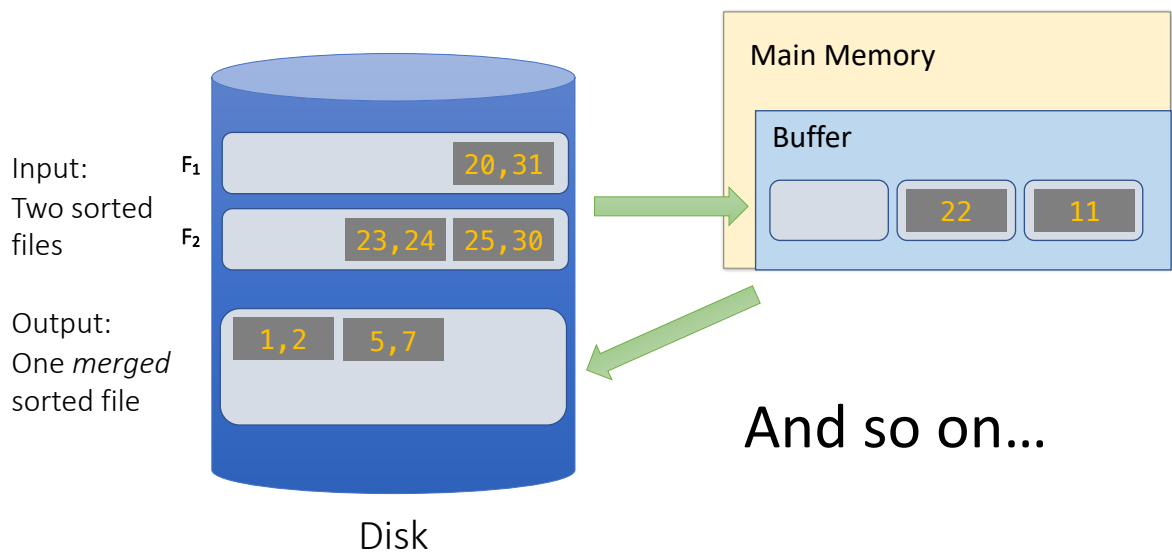
56

## External Sort-Merge Algorithm



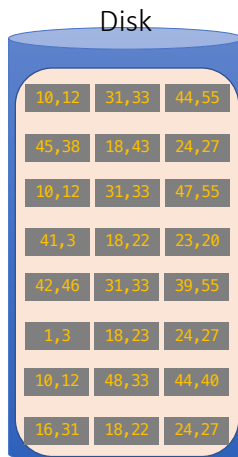
57

## External Sort-Merge Algorithm



58

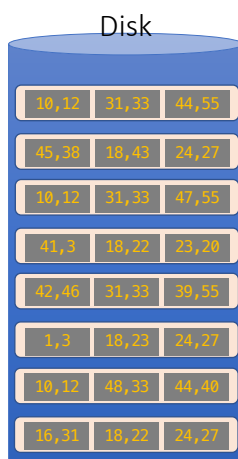
## External Sort-Merge Algorithm – larger file



Assume we still only have 3 buffer pages (*Buffer not pictured*)

59

## External Sort-Merge Algorithm – larger file



1. Split into files small enough to sort in buffer...

Assume we still only have 3 buffer pages (*Buffer not pictured*)

60

## External Sort-Merge Algorithm – larger file



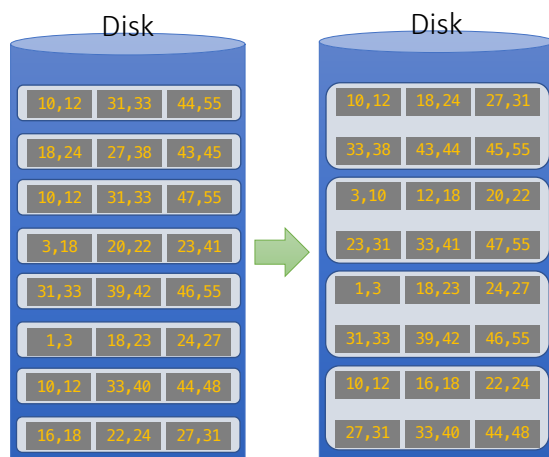
1. Split into files small enough to sort in buffer... and sort

Assume we still only have 3 buffer pages (*Buffer not pictured*)

Call each of these sorted files a **run**

61

## External Sort-Merge Algorithm – larger file

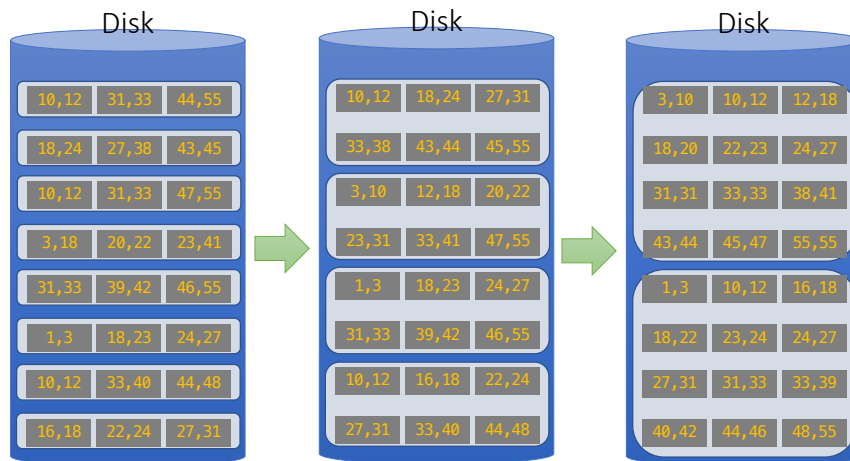


Assume we still only have 3 buffer pages (*Buffer not pictured*)

2. Now merge pairs of (sorted) files... **the resulting files will be sorted!**

62

## External Sort-Merge Algorithm – larger file



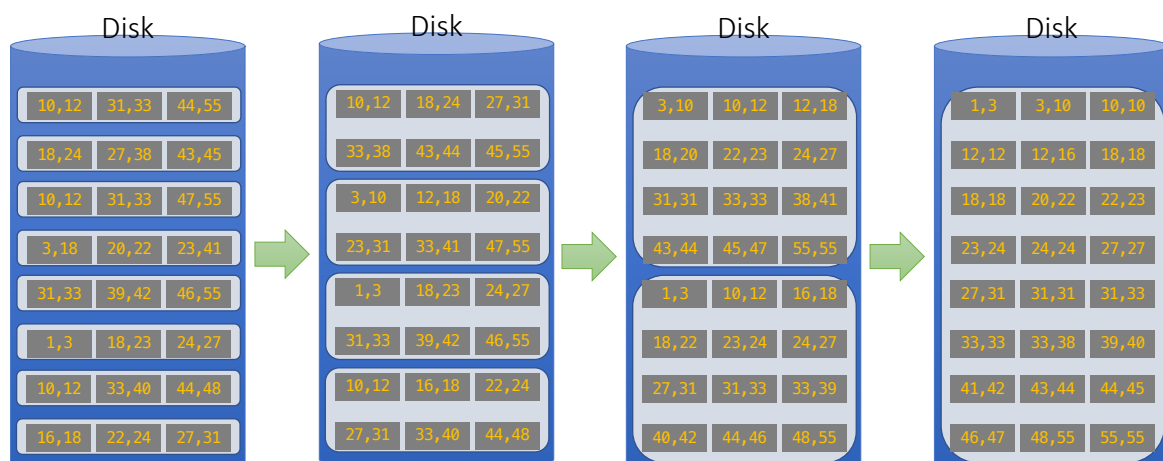
Assume we still only have 3 buffer pages (*Buffer not pictured*)

3. And repeat...

Call each of these steps a *pass*

63

## External Sort-Merge Algorithm – larger file



4. And repeat!

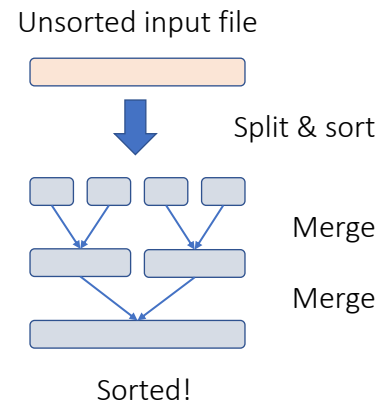
64

## Simplified 3-page Buffer Version

Assume for simplicity that we split an  $N$ -page file into  $N$  single-page **runs** and sort these; then:

- First pass: Merge  $N/2$  **pairs of runs** each of length **1 page**
- Second pass: Merge  $N/4$  **pairs of runs** each of length **2 pages**
- In general, for  $N$  pages, we do  $\lceil \log_2 N \rceil$  passes
  - +1 for the initial split & sort
- Each pass involves reading in & writing out all the pages =  $2N$  IO

→  $2N * (\lceil \log_2 N \rceil + 1)$  total IO cost!



65

## Using $B+1$ buffer pages to reduce # of passes

Suppose we have  $B+1$  buffer pages now; we can:

1. **Increase length of initial runs.** Sort  $B+1$  at a time!

At the beginning, we can split the  $N$  pages into runs of length  $B+1$  and sort these in memory

IO Cost:

$$2N(\lceil \log_2 N \rceil + 1)$$



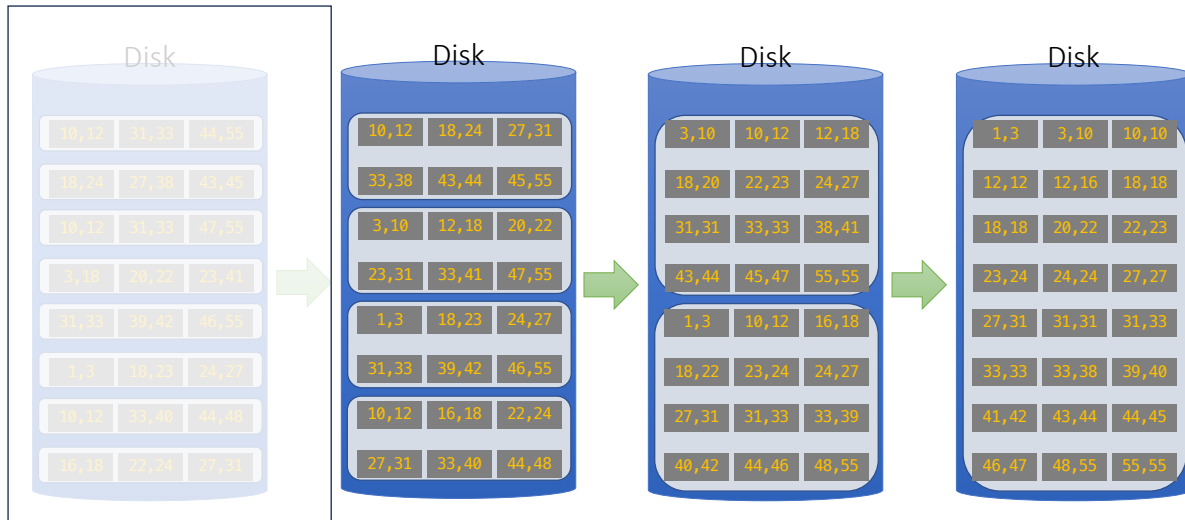
$$2N\left(\left\lceil \log_2 \frac{N}{B+1} \right\rceil + 1\right)$$

Starting with runs  
of length 1

Starting with runs of  
length  $B+1$

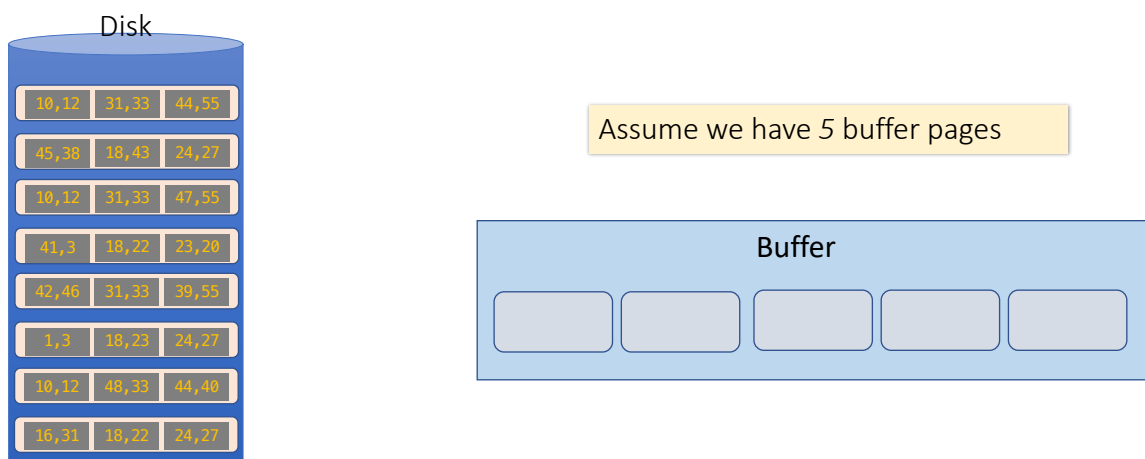
66

## External Sort-Merge Algorithm – 7 buffer pages



67

## External Sort-Merge Algorithm – B-way merge



68

## External Sort-Merge Algorithm – B-way merge

### 3 buffer pages

Unsorted input file



Split & sort



Merge



Merge



Sorted!

### 5 buffer pages

Unsorted input file



Split & sort



Merge



Sorted!

One Merge less!!!

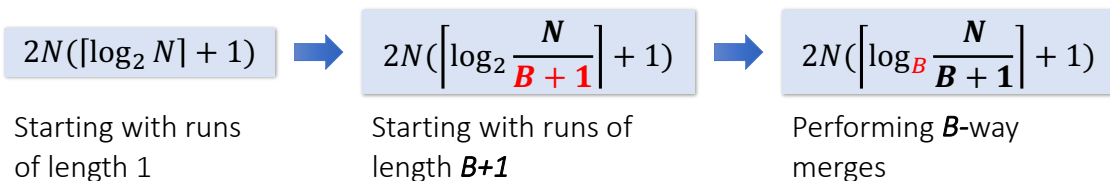
69

## Using B+1 buffer pages to reduce # of passes

### 2. Perform a B-way merge.

On each pass, we can merge groups of  $B$  runs at a time (vs. merging pairs of runs)!

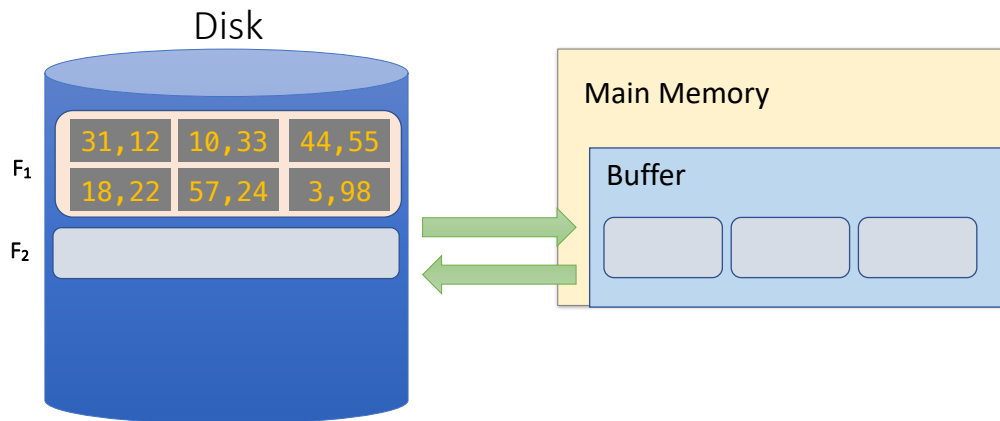
IO Cost:



70

## External Sort-Merge Algorithm – Repacking

- **Repacking** – longer initial runs by “merging” in buffer as we sort at initial stage
- Start with unsorted single input file, and load 2 pages

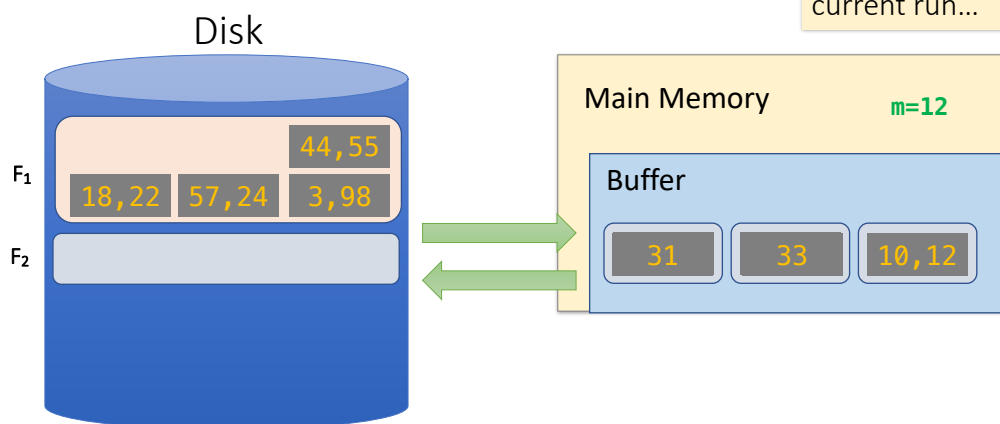


71

## External Sort-Merge Algorithm – Repacking

- Take the minimum two values, and put in output page

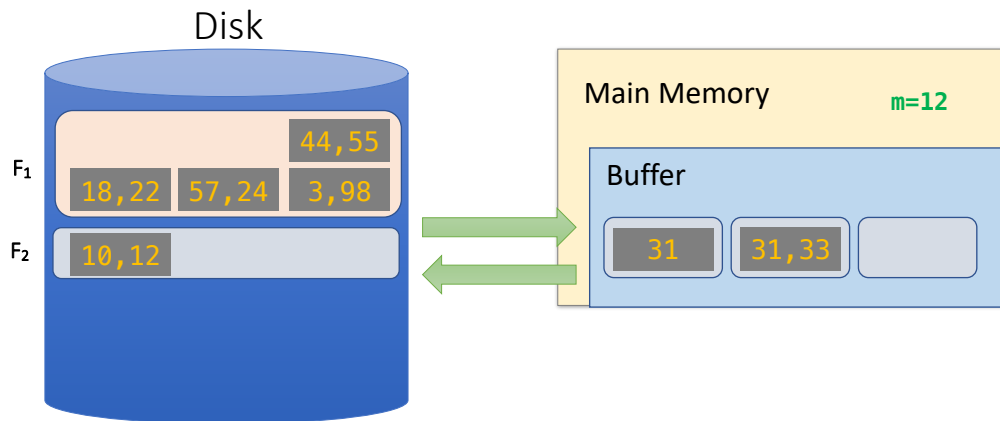
Also keep track of  
max (last) value in  
current run...



72

## External Sort-Merge Algorithm – Repacking

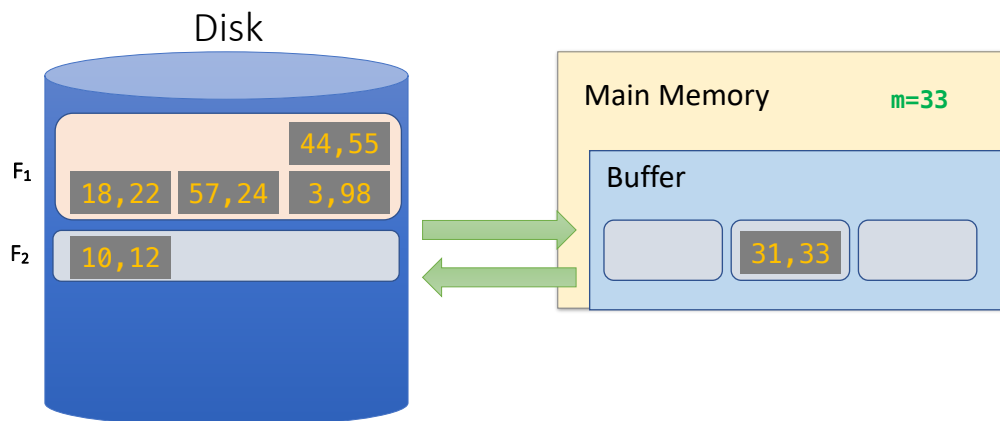
- Next, **repack**



73

## External Sort-Merge Algorithm – Repacking

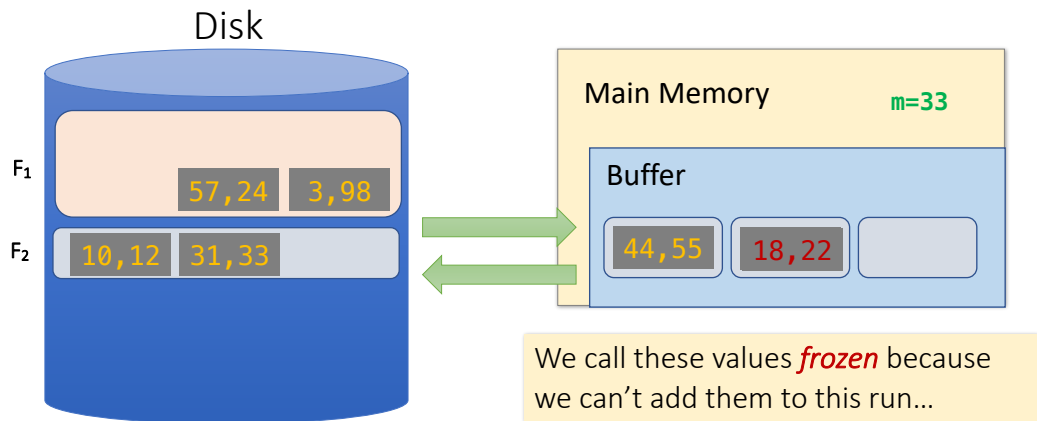
- Next, **repack**, then load another page and continue!



74

## External Sort-Merge Algorithm – Repacking

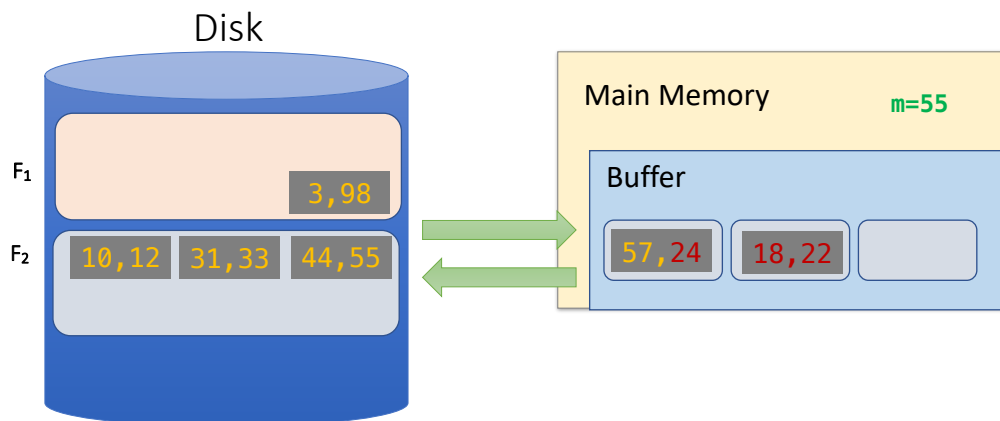
- Now, however, **the smallest values are less than the largest (last) in the sorted run...**



75

## External Sort-Merge Algorithm – Repacking

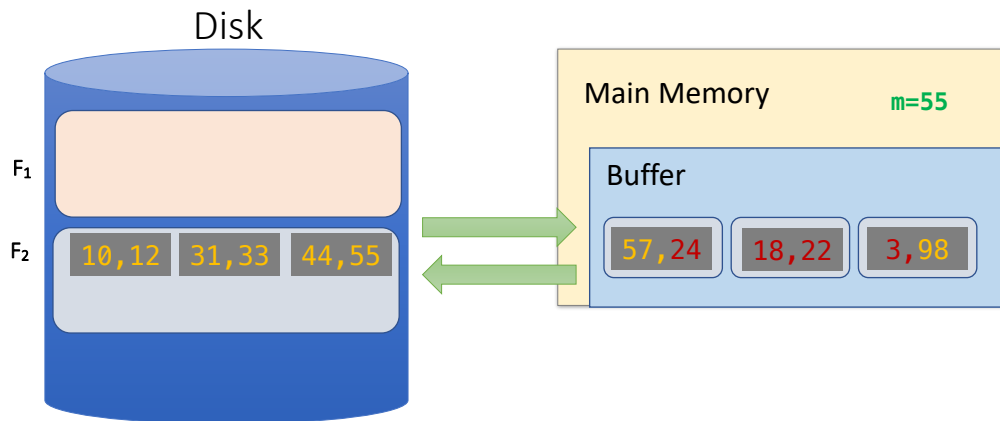
- Now, however, **the smallest values are less than the largest (last) in the sorted run...**



76

## External Sort-Merge Algorithm – Repacking

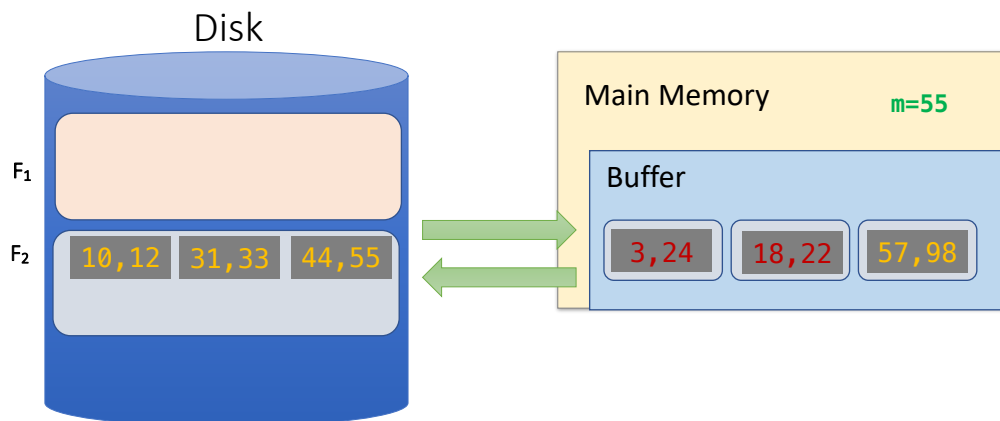
- Now, however, **the smallest values are less than the largest (last) in the sorted run...**



77

## External Sort-Merge Algorithm – Repacking

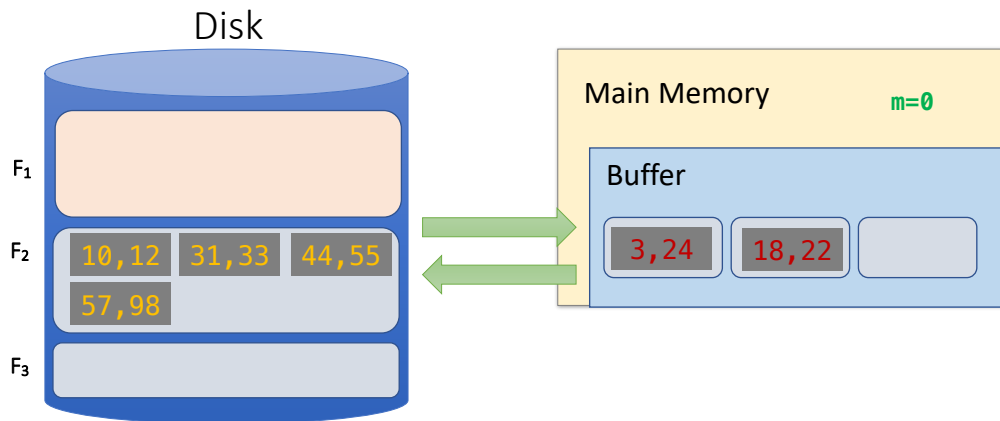
- Now, however, **the smallest values are less than the largest (last) in the sorted run...**



78

## External Sort-Merge Algorithm – Repacking

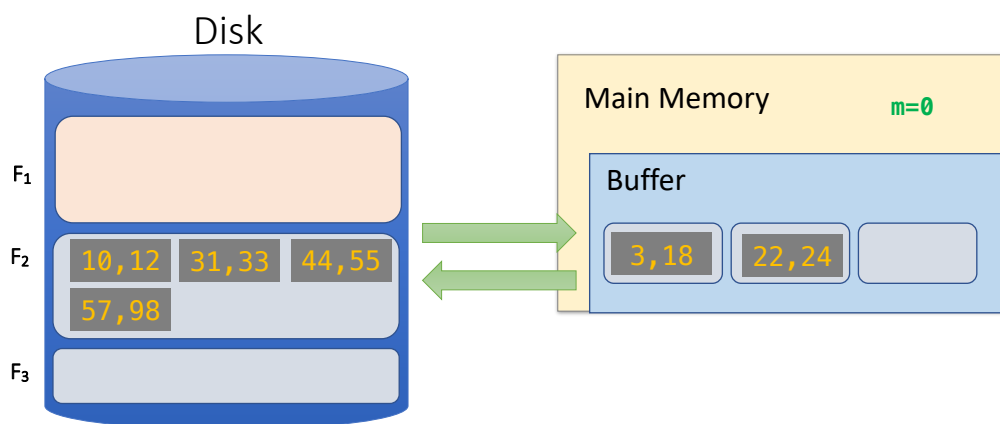
- Once **all buffer pages have a frozen value**, or input file is empty, start new run with the frozen values



79

## External Sort-Merge Algorithm – Repacking

- Once **all buffer pages have a frozen value**, or input file is empty, start new run with the frozen values



80

## External Sort-Merge Algorithm – Repacking

- Note that, for buffer with  $B+1$  pages:
  - If input file is sorted  $\rightarrow$  nothing is frozen  $\rightarrow$  we get **a single** run!
  - If input file is reverse sorted (worst case)  $\rightarrow$  everything is frozen  $\rightarrow$  we get runs of length  **$B+1$**
- In general, with repacking we do **no worse** than without it!
- What if the file is already sorted?
- Engineer's approximation: runs will have  **$\sim 2(B+1)$**  length

$$\sim 2N \left( \left\lceil \log_B \frac{N}{2(B+1)} \right\rceil + 1 \right)$$

81

## Algorithms for computing the join of relations

1. Nested Loop Join (NLJ)
2. Block Nested Loop Join (BNLJ)
3. Index Nested Loop Join (INLJ)
4. Sort-Merge Join (SMJ)
5. Hash Join (HJ)

82

## Nested Loop Join (NLJ)

Compute  $R \bowtie S$  on  $A$ :

```
for r in R:
    for s in S:
        if r[A] == s[A]:
            yield (r,s)
```

Given a relation  $R$ , let:

$T(R)$  = number of tuples in  $R$

$P(R)$  = number of pages in  $R$

Cost:

$P(R)$

**1. Loop over the tuples in  $R$**

Note that our IO cost is based on the number of *pages* loaded, not the number of tuples!

83

## Nested Loop Join (NLJ)

Compute  $R \bowtie S$  on  $A$ :

```
for r in R:
    for s in S:
        if r[A] == s[A]:
            yield (r,s)
```

Given a relation  $R$ , let:

$T(R)$  = number of tuples in  $R$

$P(R)$  = number of pages in  $R$

Cost:

$P(R) + T(R) * P(S)$

**1. Loop over the tuples in  $R$**

**2. For every tuple in  $R$ , loop over all the tuples in  $S$**

Have to read *all of  $S$*  from disk for *every tuple in  $R$* !

84

## Nested Loop Join (NLJ)

```

Compute  $R \bowtie S$  on  $A$ :
  for  $r$  in  $R$ :
    for  $s$  in  $S$ :
      if  $r[A] == s[A]$ :
        yield  $(r,s)$ 

```

Given a relation  $R$ , let:

$T(R)$  = number of tuples in  $R$

$P(R)$  = number of pages in  $R$

Cost:

$$P(R) + T(R) * P(S)$$

1. Loop over the tuples in  $R$
2. For every tuple in  $R$ , loop over all the tuples in  $S$
3. Check against join conditions

Note that NLJ can handle things other than equality constraints...

85

## Nested Loop Join (NLJ)

```

Compute  $R \bowtie S$  on  $A$ :
  for  $r$  in  $R$ :
    for  $s$  in  $S$ :
      if  $r[A] == s[A]$ :
        yield  $(r,s)$ 

```

What would *OUT* be if our join condition is trivial (if *TRUE*)?

*OUT* could be bigger than  $P(R) * P(S)$ ... but usually not that bad

Cost:

$$P(R) + T(R) * P(S) + OUT$$

1. Loop over the tuples in  $R$
2. For every tuple in  $R$ , loop over all the tuples in  $S$
3. Check against join conditions
4. Write out (to page, then when page full, to disk)

86

## Nested Loop Join (NLJ)

$T(R) = 10\,000$  tuples – **big table**

$P(R) = 100$  pages (100 tuples per page)

$T(S) = 1\,000$  tuples – **small table**

$P(S) = 50$  pages (20 tuples per page)

$P(R) + T(R) * P(S) = 100 + 10\,000 * 50 = \mathbf{500\,100}$

$P(S) + T(S) * P(R) = 50 + 1\,000 * 100 = \mathbf{100\,050}$

Given a relation R, let:

$T(R)$  = number of tuples in R

$P(R)$  = number of pages in R

Cost:

$P(R) + T(R) * P(S) + \text{OUT}$

What if R (“outer”) and S (“inner”) switched?



$P(S) + T(S) * P(R) + \text{OUT}$

Outer vs. inner selection makes a huge difference – DBMS knows which relation is smaller!

87

## Block Nested Loop Join (BNLJ)

Given  **$B+1$**  pages of memory

Cost:

$P(R)$

Compute  $R \bowtie S$  on A:

for each  $B-1$  pages  $pr$  of R:

for page  $ps$  of S:

for each tuple  $r$  in  $pr$ :

for each tuple  $s$  in  $ps$ :

if  $r[A] == s[A]$ :

yield  $(r,s)$

**1. Load in  $B-1$  pages of R at a time (leaving 1 page each free for S & output)**

*Note: There could be some speedup here due to the fact that we’re reading in multiple pages sequentially however we’ll ignore this here!*

88

## Block Nested Loop Join (BNLJ)

Given  $B+1$  pages of memory

Cost:

Compute  $R \bowtie S$  on  $A$ :

```
for each B-1 pages pr of R:
  for page ps of S:
    for each tuple r in pr:
      for each tuple s in ps:
        if r[A] == s[A]:
          yield (r,s)
```

$$P(R) + \frac{P(R)}{B-1}P(S)$$

1. Load in B-1 pages of R at a time (leaving 1 page each free for S & output)
2. For each (B-1)-page segment of R, load each page of S

Note: Faster to iterate over the *smaller* relation first!

89

## Block Nested Loop Join (BNLJ)

Given  $B+1$  pages of memory

Cost:

Compute  $R \bowtie S$  on  $A$ :

```
for each B-1 pages pr of R:
  for page ps of S:
    for each tuple r in pr:
      for each tuple s in ps:
        if r[A] == s[A]:
          yield (r,s)
```

$$P(R) + \frac{P(R)}{B-1}P(S)$$

1. Load in B-1 pages of R at a time (leaving 1 page each free for S & output)
2. For each (B-1)-page segment of R, load each page of S
3. Check against the join conditions

BNLJ can also handle non-equality constraints

90

## Block Nested Loop Join (BNLJ)

Given  $B+1$  pages of memory

Compute  $R \bowtie S$  on  $A$ :

```
for each B-1 pages pr of R:
  for page ps of S:
    for each tuple r in pr:
      for each tuple s in ps:
        if r[A] == s[A]:
          yield (r,s)
```

Again, *OUT* could be bigger than  $P(R)*P(S)$ ... but usually not that bad

Cost:

$$P(R) + \frac{P(R)}{B-1} P(S) + OUT$$

1. Load in B-1 pages of R at a time (leaving 1 page each free for S & output)
2. For each (B-1)-page segment of R, load each page of S
3. Check against the join conditions

4. Write out

91

## BNLJ vs. NLJ: Benefits of IO Aware

- In BNLJ, by loading larger chunks of R, we minimize the number of full *disk reads* of S
  - We only read all of S from disk for **every (B-1)-page segment of R!**
  - Still the full cross-product, but more done only *in memory*

NLJ

$$P(R) + T(R) * P(S) + OUT$$



BNLJ

$$P(R) + \frac{P(R)}{B-1} P(S) + OUT$$

BNLJ is faster by roughly  $\frac{(B-1)T(R)}{P(R)}$  !

92

## BNLJ vs. NLJ: Benefits of IO Aware

- Example:
  - P(R): 500 pages
  - P(S): 1000 pages
  - 100 tuples / page
  - We have 12 pages of memory (B = 11)
- NLJ: Cost = 500 + **50,000\*1000 = 50 Million IOs**
- BNLJ: Cost = 500 +  $\frac{500*1000}{10}$  = **50 Thousand IOs**

*Ignoring OUT here...*

A very real difference from a small change in the algorithm!

93

## Index Nested Loop Join (INLJ)

```

Compute R ⋈ S on A:
  Given index idx on S.A:
    for r in R:
      s in idx(r[A]):
        yield r,s
  
```

Cost:

$$P(R) + T(R)*L + OUT$$

where  $L$  is the IO cost to access all the distinct values in the index; assuming these fit on one page,  $L \sim 3$  is good est.

→ We can use an **index** (e.g. B<sup>+</sup> Tree) to **avoid doing the full cross-product!**

94

## Sort Merge Join (SMJ): Basic Procedure

To compute  $R \bowtie_A S$ :

1. Sort  $R, S$  on  $A$  using **external merge sort**
2. **Scan** sorted files and “merge”
3. [May need to “backup”]

Note that we are only considering equality join conditions here

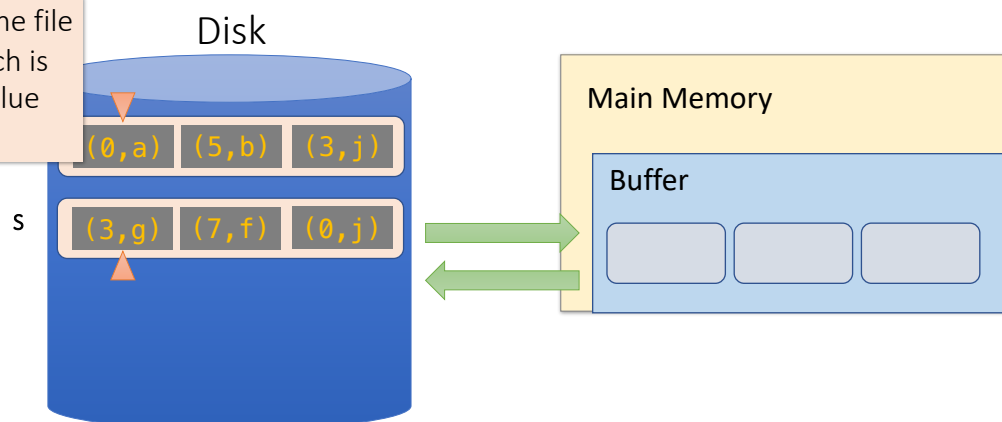
If  $R, S$  are already sorted on  $A$ , SMJ will be very fast!

95

## SMJ Example: $R \bowtie S$ on $A$ with 3 page buffer

- For simplicity: Let each page be **one tuple**, and let the first value be  $A$

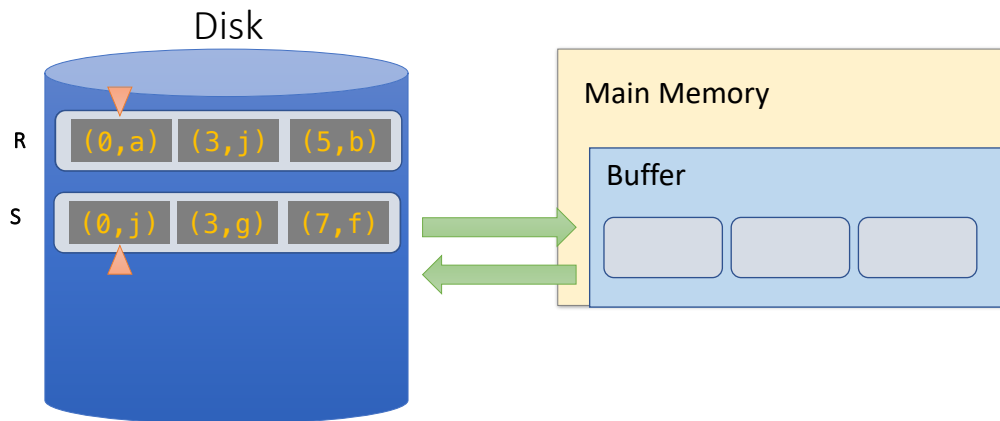
We show the file HEAD, which is the next value to be read!



96

## SMJ Example: $R \bowtie S$ on $A$ with 3 page buffer

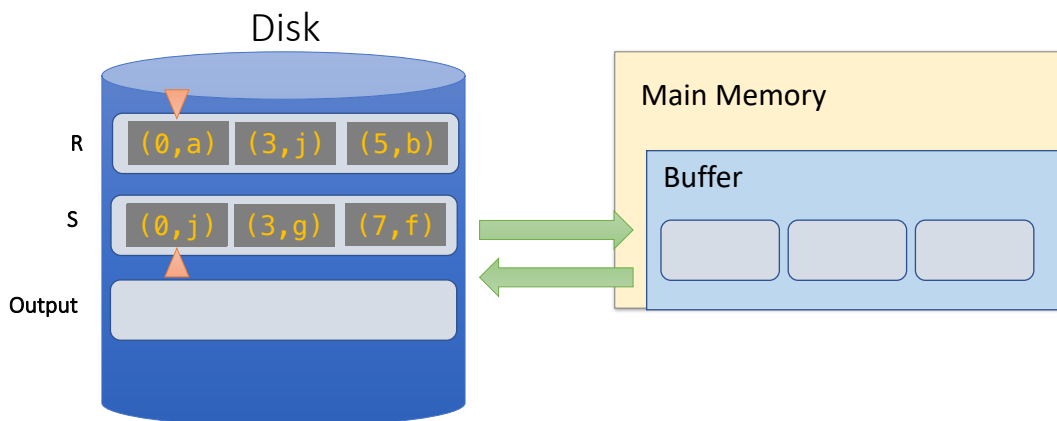
1. Sort the relations  $R, S$  on the join key (first value)



97

## SMJ Example: $R \bowtie S$ on $A$ with 3 page buffer

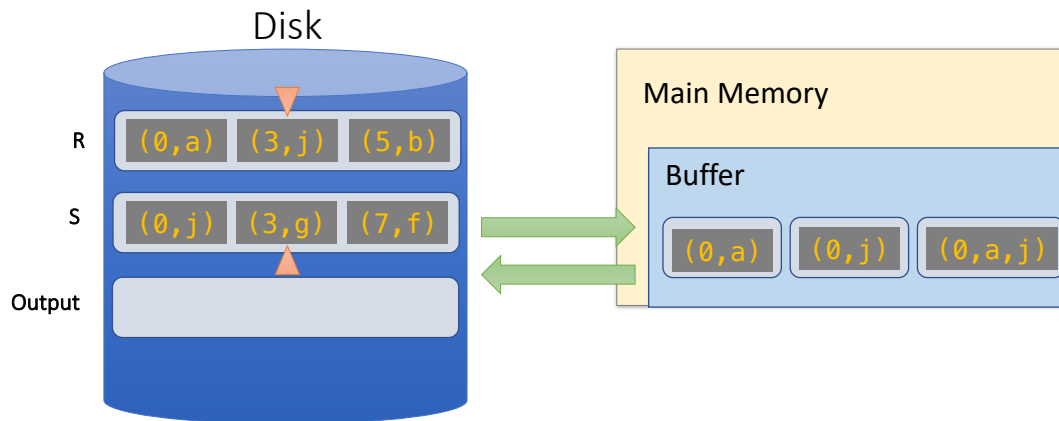
2. Scan and “merge” on join key!



98

## SMJ Example: $R \bowtie S$ on $A$ with 3 page buffer

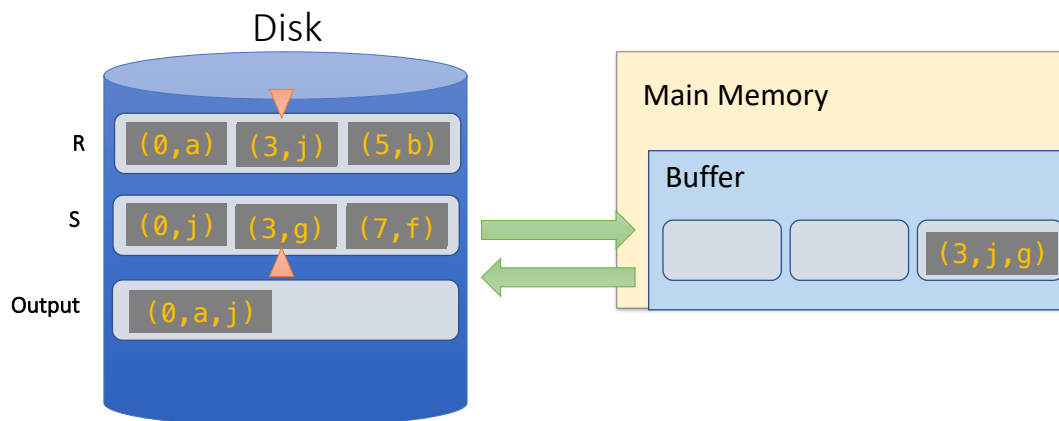
2. Scan and “merge” on join key!



99

## SMJ Example: $R \bowtie S$ on $A$ with 3 page buffer

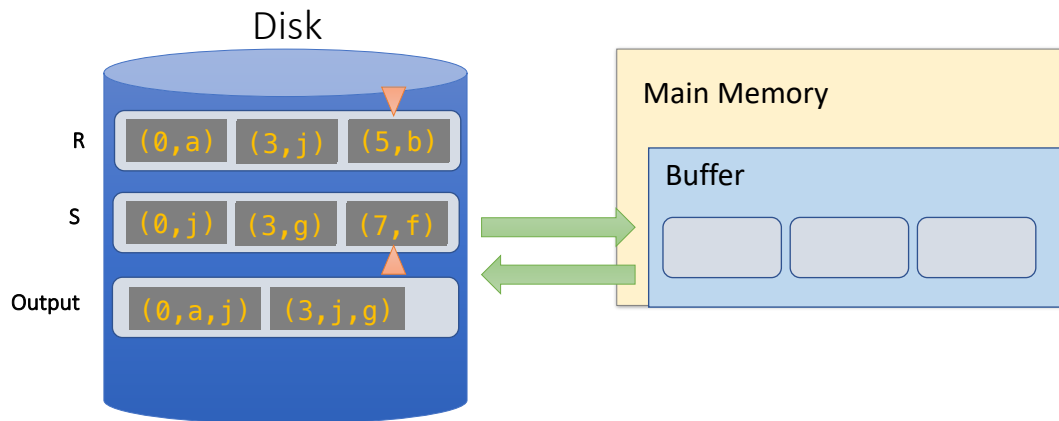
2. Scan and “merge” on join key!



100

## SMJ Example: $R \bowtie S$ on $A$ with 3 page buffer

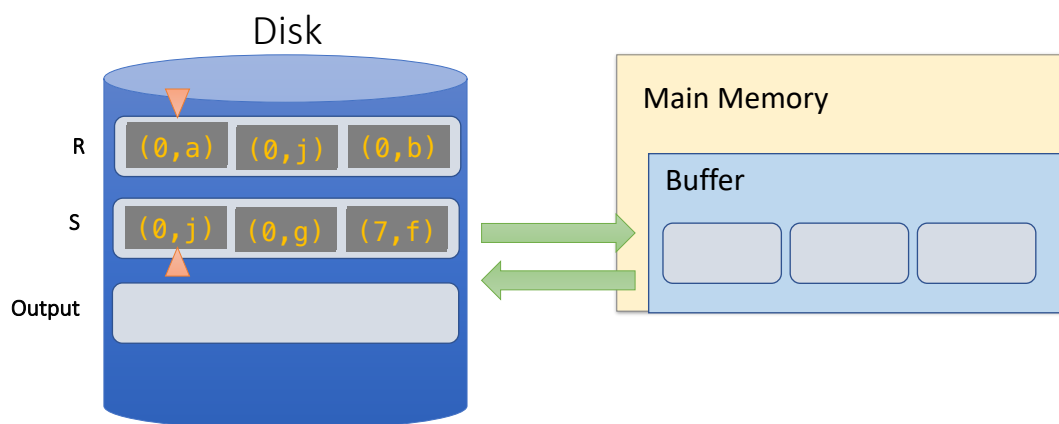
2. Done!



101

## Multiple tuples with Same Join Key: “Backup”

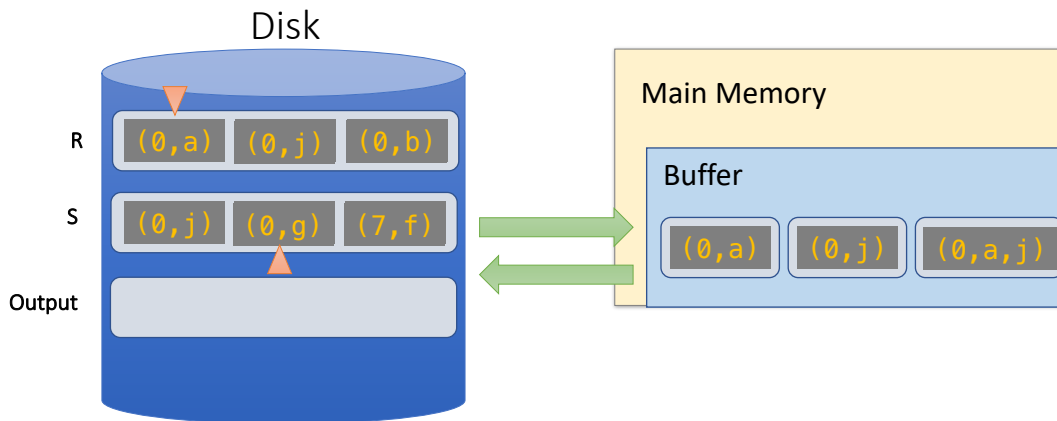
1. Start with sorted relations, and begin scan / merge...



102

## Multiple tuples with Same Join Key: “Backup”

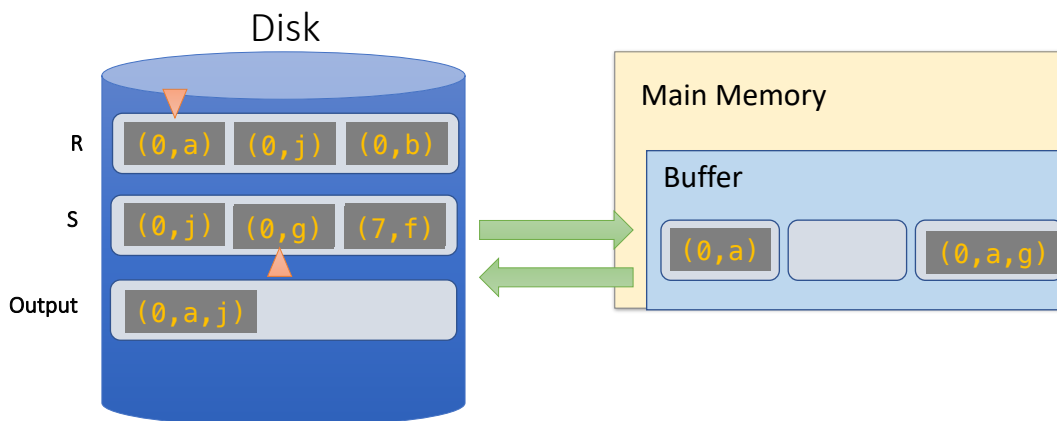
1. Start with sorted relations, and begin scan / merge...



103

## Multiple tuples with Same Join Key: “Backup”

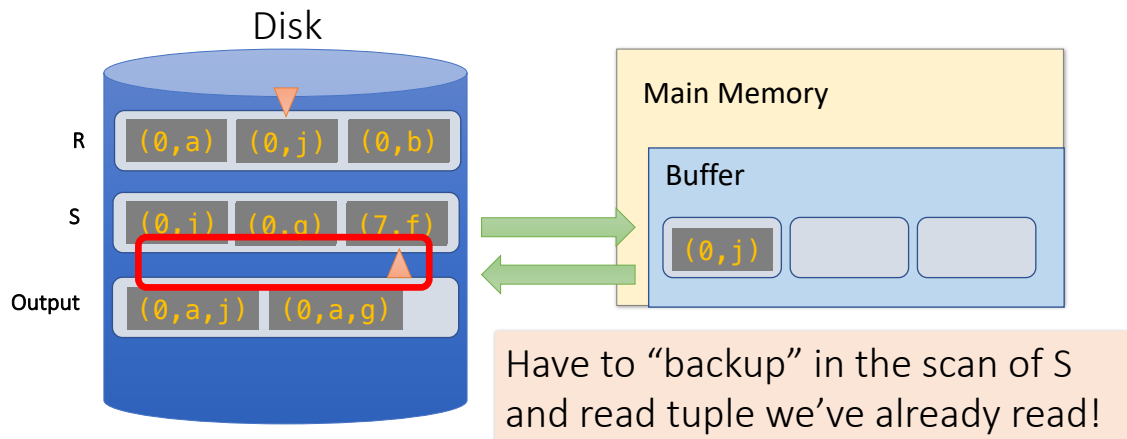
1. Start with sorted relations, and begin scan / merge...



104

## Multiple tuples with Same Join Key: “Backup”

1. Start with sorted relations, and begin scan / merge...



105

## Backup

- At best, no backup  $\rightarrow$  scan takes  $P(R) + P(S)$  reads
  - For ex: if no duplicate values in join attribute
- At worst (e.g. full backup each time), scan could take  $P(R) * P(S)$  reads!
  - For ex: if *all* duplicate values in join attribute, i.e. all tuples in R and S have the same value for the join attribute
  - Roughly: For each page of R, we'll have to *back up* and read each page of S...
- Often not that bad however, plus we can:
  - Leave more data in buffer (for larger buffers)
  - Can “zig-zag” (see animation)

106

## SMJ: Total cost

- Cost of SMJ is **cost of sorting** R and S...
- Plus the **cost of scanning**:  $\sim P(R) + P(S)$ 
  - Because of *backup*: in worst case  $P(R) * P(S)$ ; but this would be very unlikely
- Plus the **cost of writing out**:  $\sim P(R) + P(S)$  but in worst case  $T(R) * T(S)$

$$\sim \text{Sort}(P(R)) + \text{Sort}(P(S)) + P(R) + P(S) + \text{OUT}$$

$$\text{Recall: } \text{Sort}(N) \approx 2N \left( \left\lceil \log_B \frac{N}{2(B+1)} \right\rceil + 1 \right)$$

*Note: this is using repacking, where we estimate that we can create initial runs of length  $\sim 2(B+1)$*

107

## SMJ vs. BNLJ

- If we have 100 buffer pages,  $P(R) = 1000$  pages and  $P(S) = 500$  pages:
  - Sort both in two passes:  $2 * 2 * 1000 + 2 * 2 * 500 = \mathbf{6,000 \text{ IOs}}$
  - Merge phase  $1000 + 500 = 1,500 \text{ IOs}$
  - **$\approx 7,500 \text{ IOs} + \text{OUT}$**

$$\text{Sort}(P(R)) \approx 2000 \left( \left\lceil \log_{99} \frac{1000}{2 \cdot 100} \right\rceil + 1 \right) = 2000 \cdot ([0.35] + 1)$$

What is BNLJ?

$$500 + 1000 * \left\lceil \frac{500}{98} \right\rceil = \mathbf{6,500 \text{ IOs} + \text{OUT}}$$

- But, if we have 35 buffer pages?
  - Sort Merge has same behavior (still 2 passes)
  - BNLJ?  **$15,500 \text{ IOs} + \text{OUT!}$**

$$\text{Sort}(P(R)) \approx 2000 \left( \left\lceil \log_{34} \frac{1000}{2 \cdot 35} \right\rceil + 1 \right) = 2000 \cdot ([0.75] + 1)$$

SMJ is  $\sim$  linear vs. BNLJ is quadratic...  
But it's all about the memory.

108

## A Simple Optimization: Merges Merged!

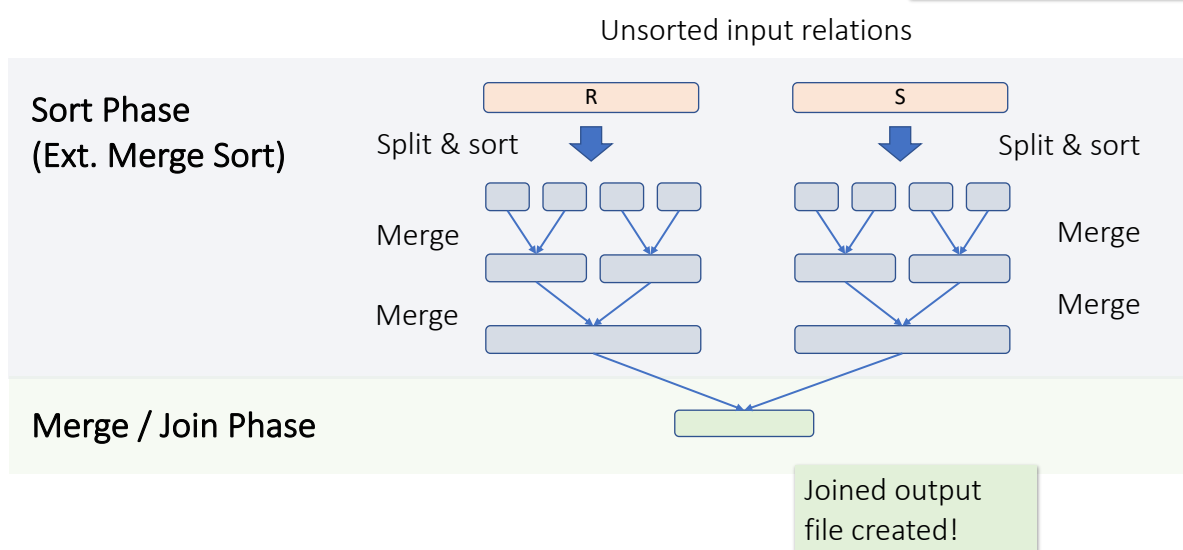
Given  $B+1$  buffer pages

- SMJ is composed of a **sort phase** and a **merge phase**
- During the **sort phase**, run passes of external merge sort on R and S
  - Suppose at some point, R and S have  $\leq B$  (sorted) runs in total
  - We could do two merges (for each of R & S) at this point, complete the sort phase, and start the merge phase...
  - OR, we could combine them: do **one** B-way merge and complete the join!

109

## Un-Optimized SMJ

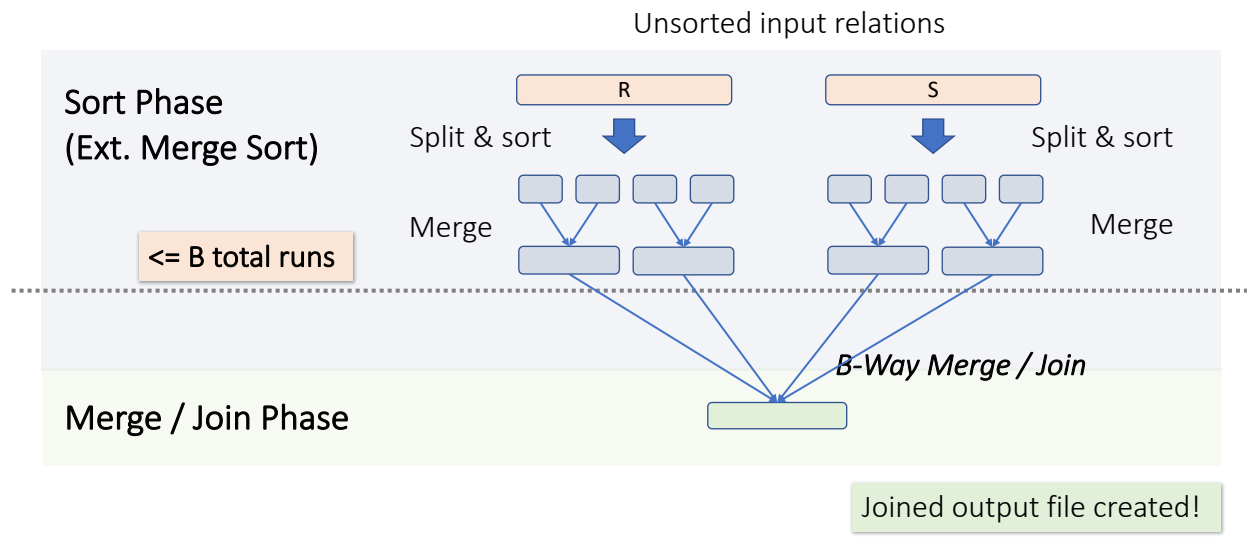
Given  $B+1$  buffer pages



110

## Simple SMJ Optimization

Given  $B+1$  buffer pages



111

## Simple SMJ Optimization

Given  $B+1$  buffer pages

- Now, on this last pass, we only do  $P(R) + P(S)$  IOs to complete the join!
- If we can initially split R and S into **B total runs each of length approx.  $\leq 2(B+1)$** , assuming repacking lets us create initial runs of  $\sim 2(B+1)$ - then we only need  **$3(P(R) + P(S)) + OUT$**  for SMJ!
  - 2 R/W per page to sort runs in memory, 1 R per page to B-way merge / join!
- How much memory for this to happen?
  - $\frac{P(R)+P(S)}{B} \leq 2(B+1) \Rightarrow \sim P(R) + P(S) \leq 2B^2$
  - Thus,  $\max\{P(R), P(S)\} \leq B^2$  is an approximate sufficient condition

If the larger of R,S has  $\leq B^2$  pages, then SMJ costs  **$3(P(R)+P(S)) + OUT!$**

112

## Takeaway points from SMJ

If input already sorted on join key, skip the sorts.

- SMJ is basically linear.
- Nasty but unlikely case: Many duplicate join keys.

SMJ needs to sort **both** relations

- If  $\max \{ P(R), P(S) \} < B^2$  then cost is  $3(P(R)+P(S)) + OUT$

113

## Recall: Hashing

- A hash function  $h_B$  maps into  $[0, B-1]$
- And maps nearly uniformly
- A hash **collision** is when  $x \neq y$  but  $h_B(x) = h_B(y)$ 
  - Note however that it will **never** occur that  $x = y$  but  $h_B(x) \neq h_B(y)$
- We hash on an attribute  $A$ , so our hash function is  $h_B(t)$  has the form  $h_B(t.A)$ .
  - **Collisions** may be more frequent.

114

## Hash Join: High-level procedure

To compute  $R \bowtie S$  on  $A$ :

Note again that we are only considering equality constraints here

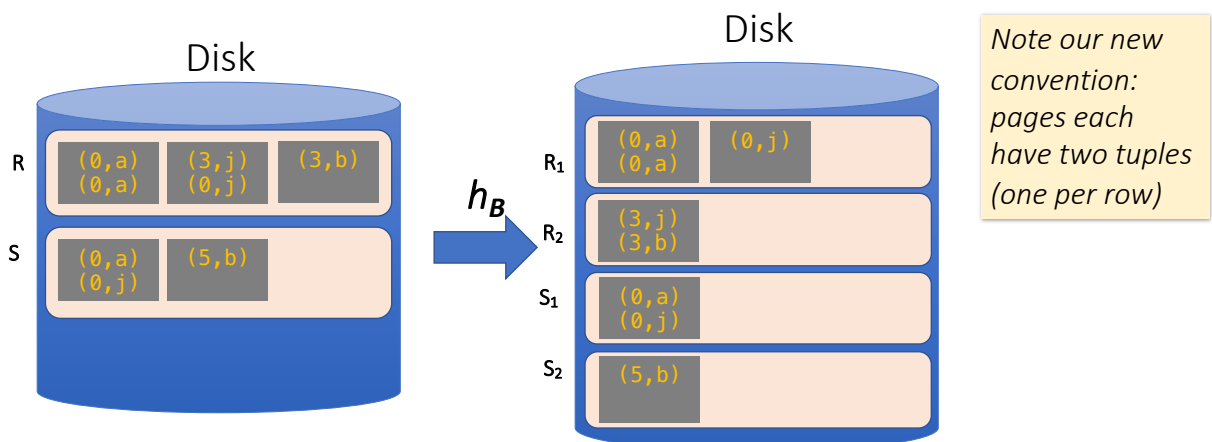
1. **Partition Phase:** Using one (shared) hash function  $h_B$ , partition  $R$  and  $S$  into  $B$  buckets
2. **Matching Phase:** Take pairs of buckets whose tuples have the same values for  $h$ , and join these
  1. Use BNLJ here; or hash again  $\rightarrow$  either way, operating on small partitions so fast!

We *decompose* the problem using  $h_B$ , then complete the join

115

## Hash Join: High-level procedure

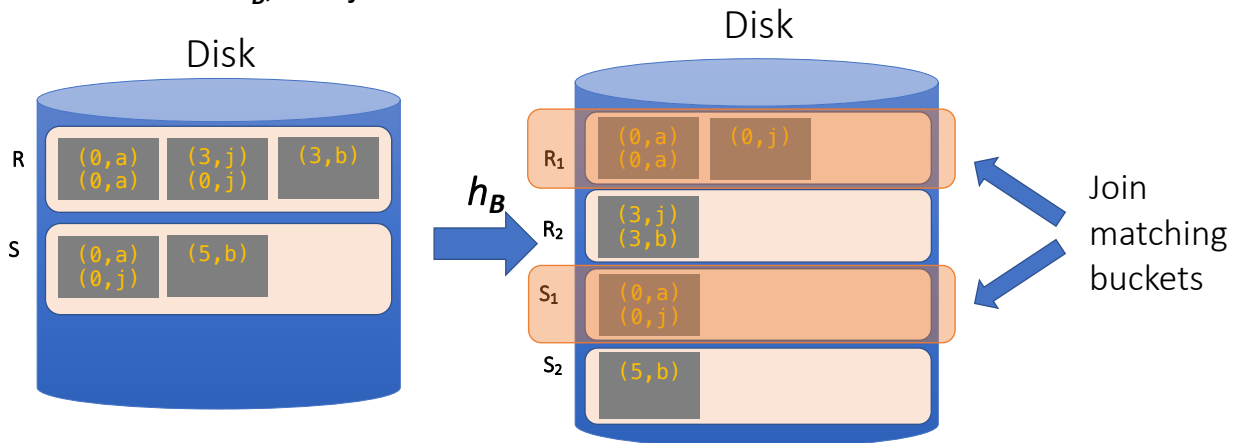
1. **Partition Phase:** Using one (shared) hash function  $h_B$ , partition  $R$  and  $S$  into  $B$  buckets



116

## Hash Join: High-level procedure

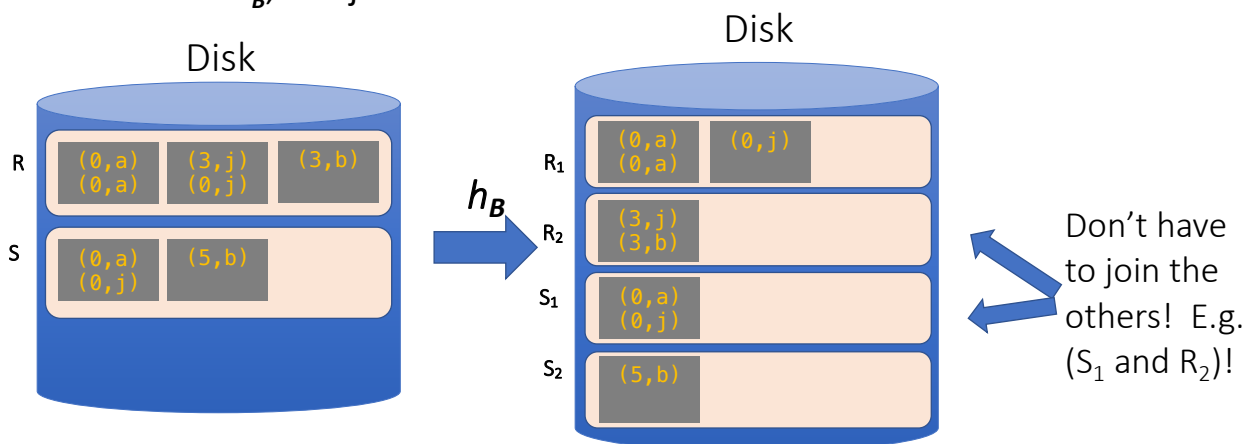
**2. Matching Phase:** Take pairs of buckets whose tuples have the same values for  $h_B$ , and join these



117

## Hash Join: High-level procedure

**2. Matching Phase:** Take pairs of buckets whose tuples have the same values for  $h_B$ , and join these



118

## Hash Join Phase 1: Partitioning

**Goal:** For each relation, partition relation into **buckets** such that if  $h_B(t.A) = h_B(t'.A)$  they are in the same bucket

Given  $B+1$  buffer pages, we partition into  $B$  buckets:

- We use  $B$  buffer pages for output (one for each bucket), and 1 for input
  - The “dual” of sorting.
  - For each tuple  $t$  in input, copy to buffer page for  $h_B(t.A)$
  - When page fills up, flush to disk.

119

## How big are the resulting buckets?

Given  **$B+1$**  buffer pages

- Given  **$N$  input pages, we partition into  $B$  buckets:**
- → Ideally our buckets are each of size  $\sim N/B$  pages
- What happens if there are **hash collisions**?
  - Buckets could be  $> N/B$
  - **We'll do several passes...**
- What happens if there are **duplicate join keys**?
  - Nothing we can do here... could have some **skew** in size of the buckets

120

## How big *do we want* the resulting buckets?

Given  **$B+1$**  buffer pages

- Ideally, our buckets would be of size  $\leq B - 1$  pages
  - 1 for input page, 1 for output page,  $B-1$  for each bucket
- Recall: If we want to join a bucket from R and one from S, we can do BNLJ **in linear time** if for *one of them*,  $P(R) \leq B - 1$ !
  - And more generally, being able to fit bucket in memory is advantageous
- We can keep partitioning buckets that are  $> B-1$  pages, until they are  $\leq B - 1$  pages
  - Using a new hash key which will split them...

Recall for BNLJ:

$$P(R) + \frac{P(R)P(S)}{B - 1}$$

We'll call each of these a "pass" again...

121

## Hash Join Phase 2: Matching

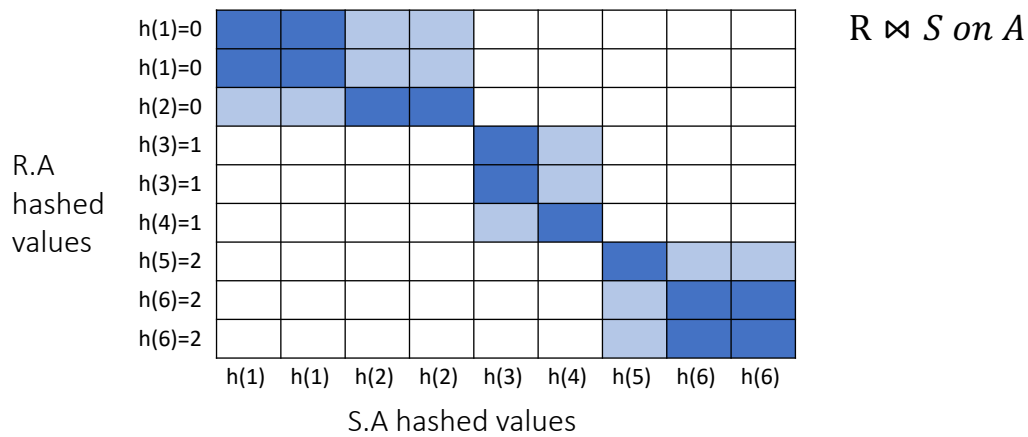
- Note that since  $x = y \rightarrow h(x) = h(y)$ , we only need to consider pairs of buckets (one from R, one from S) that have the same hash function value
- If our buckets are  $\sim B - 1$  pages, can join each such pair using BNLJ **in linear time**; recall (with  $P(R) = B-1$ ):

$$\text{BNLJ Cost: } P(R) + \frac{P(R)P(S)}{B-1} = P(R) + \frac{(B-1)P(S)}{B-1} = P(R) + P(S)$$

Joining the pairs of buckets is linear!  
(As long as smaller bucket  $\leq B-1$  pages)

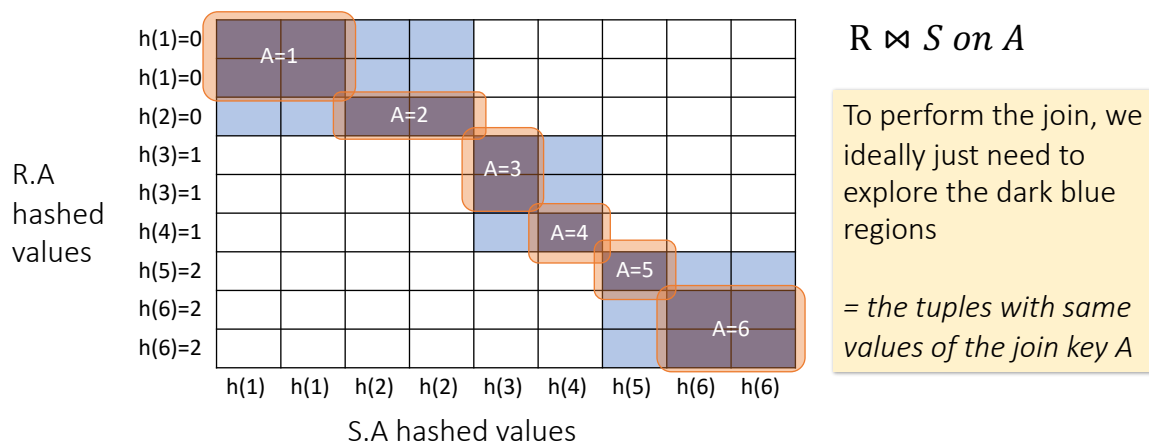
141

## Hash Join Phase 2: Matching



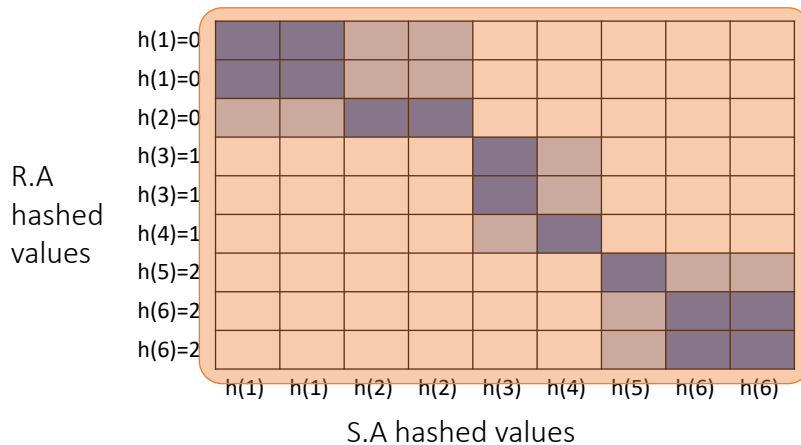
142

## Hash Join Phase 2: Matching



143

## Hash Join Phase 2: Matching

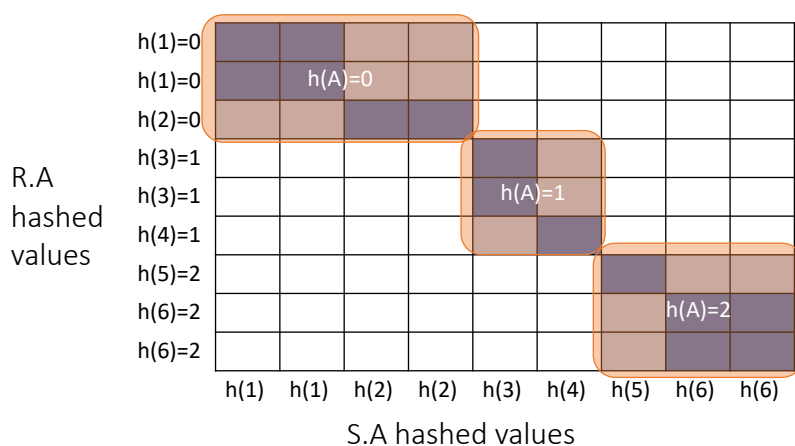


$R \bowtie S$  on  $A$

With a join algorithm like BNLJ that doesn't take advantage of equijoin structure, we'd have to explore this *whole grid!*

144

## Hash Join Phase 2: Matching



$R \bowtie S$  on  $A$

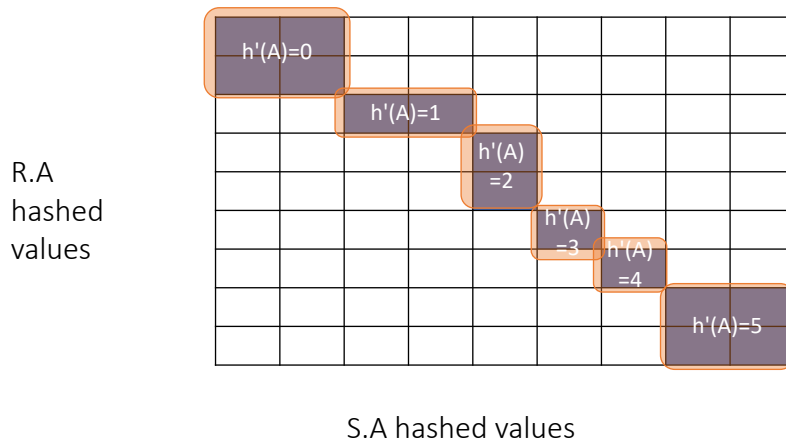
With HJ, we only explore the *blue* regions

= the tuples with same values of  $h(A)$ !

We can apply BNLJ to each of these regions

145

## Hash Join Phase 2: Matching



$R \bowtie S \text{ on } A$

An alternative to  
applying BNLJ:

We could also hash  
again and keep doing  
passes in memory to  
reduce further!

146

## How much memory do we need for HJ?

- Given  $B+1$  buffer pages Assume  $P(R) \leq P(S)$
- Suppose (reasonably) that we can partition into  $B$  buckets in 2 passes:
  - For  $R$ , we get  $B$  buckets of size  $\sim P(R)/B$
  - To join these buckets in linear time, we need these buckets to fit in  $B-1$  pages, so we have:

$$B - 1 \geq \frac{P(R)}{B} \Rightarrow \sim B^2 \geq P(R)$$

Quadratic relationship  
between *smaller*  
*relation's* size & memory!

147

## Hash Join Summary

- *Given enough buffer pages as on previous slide...*
- **Partitioning** requires reading + writing each page of R,S
  - $2(P(R)+P(S))$  IOs
- **Matching** (with BNLJ) requires reading each page of R,S
  - $P(R) + P(S)$  IOs
- **Writing out results** could be as bad as  $P(R)*P(S)$ ... but probably closer to  $P(R)+P(S)$

HJ takes  $\sim 3(P(R)+P(S)) + OUT$  IOs!

148

## Sort-Merge vs. Hash Join

- *Given enough memory*, both SMJ and HJ have performance:

$\sim 3(P(R)+P(S)) + OUT$

- *"Enough" memory =*
  - SMJ:  $B^2 > \max\{P(R), P(S)\}$
  - HJ:  $B^2 > \min\{P(R), P(S)\}$

Hash Join superior if relation sizes *differ greatly*.

149

## Additional techniques for query optimization

- **Materialization**
  - An expression can be evaluated by means of materialization, where the system computes the result of each subexpression and stores it on disk and then uses it to compute the result of the parent expression.
- **Pipelining**
  - Pipelining helps to avoid writing the results of many subexpressions to disk by using the results in the parent expression even as they are being generated.
- **Cache-Conscious Algorithms**
  - data already in CPU cache can be accessed as much as 100 times faster than data in memory.
- **Column-Oriented Storage**
  - Column stores permit efficient access to many values for a given attribute at once, they are well suited to exploit the vector-processing capabilities of modern processors.

150

## EXPLAIN in MySQL

```
EXPLAIN SELECT p.productName, p.MSRP, c.customerName, SUM( od.quantityO
rdered * od.priceEach ) AS TotalPrice FROM orders AS o
JOIN orderdetails AS od ON o.orderNumber = od.orderNumber
JOIN products AS p ON od.productCode = p.productCode
JOIN customers AS c ON o.customerNumber = c.customerNumber GROUP
BY p.productName, p.MSRP, c.customerName;
```

+ Opcje

| id | select_type | table | type   | possible_keys           | key            | key_len | ref                           | rows | Extra                           |
|----|-------------|-------|--------|-------------------------|----------------|---------|-------------------------------|------|---------------------------------|
| 1  | SIMPLE      | c     | ALL    | PRIMARY                 | NULL           | NULL    | NULL                          | 122  | Using temporary; Using filesort |
| 1  | SIMPLE      | o     | ref    | PRIMARY, customerNumber | customerNumber | 4       | pkorytkowski.c.customerNumber | 1    | Using index                     |
| 1  | SIMPLE      | od    | ref    | PRIMARY, productCode    | PRIMARY        | 4       | pkorytkowski.o.orderNumber    | 4    |                                 |
| 1  | SIMPLE      | p     | eq_ref | PRIMARY                 | PRIMARY        | 17      | pkorytkowski.od.productCode   | 1    |                                 |

151

## EXPLAIN in MS SQL Server

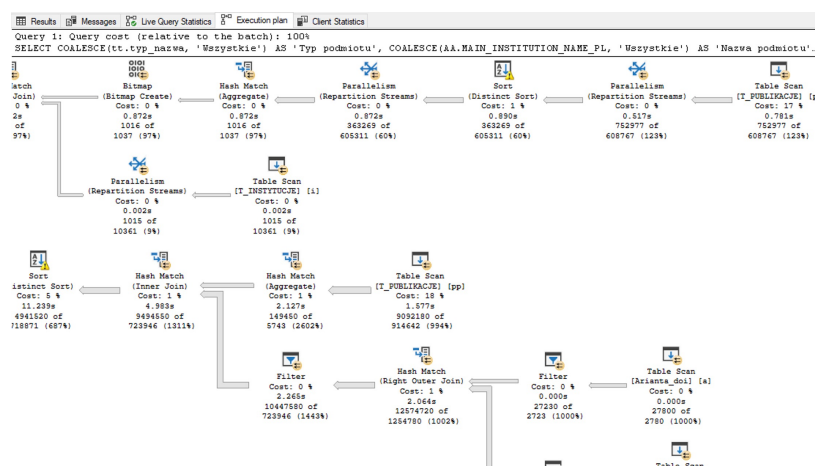
```

SELECT COALESCE(tt.typ_nazwa, 'Wszystkie') AS 'Typ podmiotu', COALESCE(AA.MAIN_INSTITUTION_NAME_PL, 'Wszystkie') AS 'Nazwa podmiotu',
SUM(AA.liczba_z_DOI) AS 'Liczba artykułów z podanym DOI', SUM(BB.liczba_mozliwych) AS 'Liczba artykułów, w których może być DOI',
CAST(ROUND(CAST((SUM(AA.liczba_z_DOI) * 100.0 / SUM(BB.liczba_mozliwych)) AS FLOAT), 2) AS NVARCHAR(10)) + '%' AS 'Procent występowania DOI'
FROM (
    SELECT i.MAIN_INSTITUTION_NAME_PL, COUNT(*) AS liczba_z_DOI
    FROM (
        SELECT DISTINCT p.INSTITUTION_UID, p.ID FROM T_PUBLIKACJE AS p WHERE (p.JOURNAL_ISSN IS NOT NULL OR p.JOURNAL_EISSN IS NOT NULL) AND p.DOI IS NOT NULL) AA
JOIN T_INSTYTUCJE AS i ON AA.INSTITUTION_UID = i.U_ID
GROUP BY i.MAIN_INSTITUTION_NAME_PL
) AA
JOIN (
    SELECT i.MAIN_INSTITUTION_NAME_PL, COUNT(*) AS liczba_mozliwych FROM (
        SELECT DISTINCT p.ID, p.INSTITUTION_UID FROM T_PUBLIKACJE AS p
        FULL JOIN Arianta_doi AS a ON a.issn = p.JOURNAL_ISSN
        WHERE EXISTS (
            SELECT pp.ID FROM T_PUBLIKACJE AS pp WHERE pp.JOURNAL_ISSN = p.JOURNAL_ISSN AND DOI IS NOT NULL
            ) AND p.JOURNAL_ISSN IS NOT NULL AND ((a.ISSN IS NOT NULL AND a.doi = 1) OR A.ISSN IS NULL)
        UNION
        SELECT p.ID, p.INSTITUTION_UID
        FROM T_PUBLIKACJE AS p
        FULL JOIN Arianta_doi AS a ON a.issn = p.JOURNAL_ISSN
        WHERE EXISTS (
            SELECT pp.ID FROM T_PUBLIKACJE AS pp WHERE pp.JOURNAL_EISSN = p.JOURNAL_EISSN AND DOI IS NOT NULL
            ) AND p.JOURNAL_EISSN IS NOT NULL AND ((a.ISSN IS NOT NULL AND a.doi = 1) OR A.ISSN IS NULL)) AA
JOIN T_INSTYTUCJE AS i ON AA.INSTITUTION_UID = i.U_ID
GROUP BY i.MAIN_INSTITUTION_NAME_PL
) BB ON AA.MAIN_INSTITUTION_NAME_PL = BB.MAIN_INSTITUTION_NAME_PL
JOIN typyUcz AS t ON t.MAIN_INSTITUTION_NAME_PL = AA.MAIN_INSTITUTION_NAME_PL
JOIN typy AS tt ON tt.typ = t.typ
GROUP BY ROLLUP (tt.typ_nazwa, AA.MAIN_INSTITUTION_NAME_PL)
ORDER BY tt.typ_nazwa, AA.MAIN_INSTITUTION_NAME_PL DESC;

```

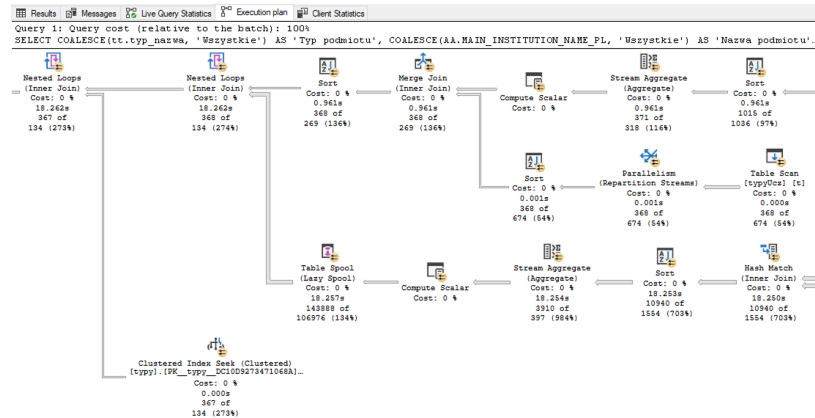
152

## EXPLAIN in MS SQL Server



153

## EXPLAIN in MS SQL Server



154

## EXPLAIN in MS SQL Server

| Rows | Executes | Stmt Text                                           | StreId | NodId | Parent | PhysicalOp       | LogicalOp        | Argument                                               |
|------|----------|-----------------------------------------------------|--------|-------|--------|------------------|------------------|--------------------------------------------------------|
| 1    | 383      | 1 SELECT COALESCE(itr_nazwa, 'Wszystkie') AS Typ... | 1      | 1     | 0      | NULL             | NULL             | NULL                                                   |
| 2    | 0        | 0  -Compute Scalar(DEFINE (Exp01027CASC WHEN P...   | 1      | 2     | 1      | Computer Scalar  | Computer Scalar  | DEFINE (Exp01027CASC WHEN (PB#N[dbo].[bpy]             |
| 3    | 0        | 0  -SortORDER BY (itr_nazwa) ASC, [[MAIN]]...       | 1      | 3     | 2      | Sort             | Sort             | ORDER BY (itr_nazwa) ASC, [[MAIN_INSTITUTIO...         |
| 4    | 0        | 0  -Compute Scalar(DEFINE (Exp01025CASC W...        | 1      | 5     | 3      | Computer Scalar  | Computer Scalar  | DEFINE (Exp01025CASC WHEN (Exp01030H) THEN             |
| 5    | 383      | 1  -Stream Aggregate(GROUP BY (itr_bpy_nazw...      | 1      | 6     | 5      | Stream Aggregate | Aggregate        | GROUP BY (itr_bpy_nazwa), [[MAIN_INSTITUTIO...         |
| 6    | 367      | 1  -Stream Aggregate(GROUP BY (itr_bpy_n...         | 1      | 7     | 6      | Stream Aggregate | Aggregate        | GROUP BY (itr_bpy_nazwa), [[MAIN_INSTITUTIO...         |
| 7    | 367      | 1  -SortORDER BY (itr_bpy_nazwa) ASC...             | 1      | 8     | 7      | Sort             | Sort             | ORDER BY (itr_bpy_nazwa), [[MAIN_INSTITUTIO...         |
| 8    | 367      | 1  -Parallel (Gather Streams)                       | 1      | 9     | 8      | Parallel         | Gather Streams   | NULL                                                   |
| 9    | 367      | 1  -Nested LoopSpinner Join, OUTE...                | 1      | 10    | 9      | Nested Loops     | Inner Join       | OUTER REFERENCES (b_bpy)                               |
| 10   | 368      | 10  -Nested LoopSpinner Join, IN...                 | 1      | 11    | 10     | Nested Loops     | Inner Join       | WHERE (PB#N[dbo].[INSTYTUCJE]) [[MAIN_INSTITU...       |
| 11   | 368      | 10  -  SortORDER BY (itr_bpy)...                    | 1      | 12    | 11     | Sort             | Sort             | ORDER BY (itr_bpy) ASC                                 |
| 12   | 368      | 10    Merge JoinInner Join...                       | 1      | 13    | 12     | Merge Join       | Inner Join       | MERGE (q) [[MAIN_INSTITUTION_NAME_P_LJ]] (q) [[MAIN... |
| 13   | 0        | 0      Compute Scalar(IDE...                        | 1      | 15    | 13     | Computer Scalar  | Computer Scalar  | DEFINE (Exp01004) CONVERT_IMPLICIT (str_foblogagg...   |
| 14   | 371      | 10        Stream Aggregate...                       | 1      | 16    | 15     | Stream Aggregate | Aggregate        | GROUP BY (q) [[MAIN_INSTITUTION_NAME_P_LJ]]            |
| 15   | 1015     | 10        SortORDER BY...                           | 1      | 17    | 16     | Sort             | Sort             | ORDER BY (q) [[MAIN_INSTITUTION_NAME_P_LJ]] ASC        |
| 16   | 1015     | 10          Bitmap PHA...                           | 1      | 18    | 17     | Bitmap Create    | Bitmap Create    | HASH (q) [[MAIN_INSTITUTION_NAME_P_LJ]]                |
| 17   | 1015     | 10          Parallel...                             | 1      | 19    | 18     | Parallelism      | Repartition Sort | PARTITION COLUMNS (q) [[INSTITUTION_NAME...            |
| 18   | 1015     | 10            Hash...                               | 1      | 20    | 19     | Hash Match       | Inner Join       | HASH (q) [[INSTITUTION_UID]] (q) [[PJ_03_RESIDUAL...   |
| 19   | 1016     | 10            Bitmap...                             | 1      | 21    | 20     | Bitmap           | Bitmap Create    | HASH (q) [[INSTITUTION_UID]]                           |
| 20   | 1016     | 10            Hash Match...                         | 1      | 22    | 21     | Hash Match       | Aggregate        | HASH (q) [[INSTITUTION_UID]], RESIDUAL (PB#N[dbo].     |
| 21   | 363269   | 10            Parallelism...                        | 1      | 23    | 22     | Parallelism      | Repartition Sort | PARTITION COLUMNS (q) [[INSTITUTION_UID]]              |
| 22   | 363269   | 10            Distinct Sort...                      | 1      | 24    | 23     | Distinct Sort    | Distinct Sort    | DISTINCT ORDER BY (q) [[INSTITUTION_UID]] ASC          |
| 23   | 75277    | 10            Parallelism...                        | 1      | 25    | 24     | Parallelism      | Table Scan       | PARTITION COLUMNS (q) [[INSTITUTION_UID]], [[PJ_03]    |
| 24   | 75277    | 10            Table Scan...                         | 1      | 26    | 25     | Table Scan       | Table Scan       | OBJECT (PB#N[dbo].[T_PUBLIKACJE]) AS q, WHERE...       |
| 25   | 1015     | 10            Parallelism...                        | 1      | 27    | 26     | Parallelism      | Repartition Sort | PARTITION COLUMNS (q) [[PJ_03]                         |
| 26   | 1015     | 10            Table Scan...                         | 1      | 28    | 27     | Table Scan       | Table Scan       | OBJECT (PB#N[dbo].[T_INSTYTUCJE]) AS j, WHERE...       |
| 27   | 368      | 10            SortORDER BY (itr_bpy)...             | 1      | 29    | 13     | Sort             | Sort             | ORDER BY (q) [[MAIN_INSTITUTION_NAME_P_LJ]] ASC        |

Query executed successfully.

PK-SQL2017PKSQL2017 (140)... PK-SQL2017\admin (53) PB#N 000020 58 row

155

## Query optimization tips

1. Add missing indexes
2. Check for unused indexes
3. Do not apply functions on indexed columns in the **WHERE** clause when there is no corresponding function-based index.

Index on myDate won't be used

**WHERE** DATEPART(YEAR, myDate) = '2021';



**WHERE** myDate >= '20210101'  
AND myDate < '20220101'

4. Avoid using multiple OR in the **WHERE** predicate

**WHERE** first\_name = 'Mia' **OR** last\_name = 'Vega';

156

## Query optimization tips

5. Use wildcards at the end of a phrase only

**WHERE** last\_name **LIKE** 'Ve%';

**WHERE** last\_name **LIKE** '%Ve%';



If necessary, use REVERSE() function.

6. Avoid too many JOINS
7. Avoid using SELECT DISTINCT
8. Use SELECT fields instead of SELECT \*
9. Use BETWEEN

**WHERE** date >= 1950 **AND** date <= 2015



**WHERE** date **BETWEEN** 1950 **AND** 2015

157

## Query optimization tips

10. Insert data in batches as a one command

```

INSERT INTO Students (ID, ...)
VALUES (12, 'Mia', 'Wallace');
INSERT INTO Students (ID, ...)
VALUES (12, 'Mia', 'Wallace'),
      (34, 'Vincent', 'Vega');
  
```



```

INSERT INTO Students (ID, ...)
VALUES (12, 'Mia', 'Wallace'),
      (34, 'Vincent', 'Vega'),
      (55, 'Butch', 'Coolidge');
  
```

11. Avoid scalar subqueries in the **SELECT** list.

```
SELECT field1, (SELECT field FROM table WHERE id = xoxox) AS field2 FROM table;
```

12. Use joins instead of subqueries.

13. Avoid mixing data types in the **WHERE** clause.

```

WHERE cust_nbr = "123"
WHERE substr(ssn,7,4) = 1234
  
```

158

## Query optimization tips

14. Avoid, whenever possible, comparison operators such as <>, NOT IN, NOT EXISTS

15. Avoid unnecessary sorts, including when **UNION ALL** rather than **UNION** is applicable.

16. Do not **HAVING** filter on not aggregated data

```

HAVING ID >= 123
  
```



```

WHERE ID >= 123
  
```

17. Use Common Table Expressions – **WITH**

More examples:

[https://andrewrgoss.com/pdf/sql\\_query\\_optimization\\_techniques.pdf](https://andrewrgoss.com/pdf/sql_query_optimization_techniques.pdf)

159

## Optimizer Hints

- A hint is a comment in a SQL statement that acts as an instruction to the optimizer.
- Hints apply on a per-statement basis, they provide finer control over statement execution plans.
- The optimizer uses hints to choose an execution plan for the statement unless prevented by some condition.
- Optimizer hints apply at different scope levels:
  - Global: The hint affects the entire statement
  - Query block: The hint affects a particular query block within a statement
  - Table-level: The hint affects a particular table within a query block
  - Index-level: The hint affects a particular index within a table

Hints are for advanced users. Use them with great caution.

160

## Optimizer Hints

- beginning of query and data change statements

```
SELECT /*+ HASH_JOIN(table1) */ ...
INSERT /*+ ... */ ...
UPDATE /*+ ... */ ...
REPLACE /*+ ORDER_INDEX(table1, idx_a) */ ...
DELETE /*+ ... */ ...
```

One hints statement

- beginning of query blocks:

```
(SELECT /*+ NO_BNL(table1) MERGE(table2) */ ... )
(SELECT ... ) UNION (SELECT /*+ ... */ ... )
(SELECT /*+ ... */ ... ) UNION (SELECT /*+ ... */ ... )
UPDATE ... WHERE x IN (SELECT /*+ ... */ ...)
INSERT ... SELECT /*+ BNL(table1)
                    JOIN_ORDER(table1, table2) */ ...
```

Multiple hints

Multiple hints

161

Thank you for your  
attention!

Time for questions

162